



SentinelOne Hyperautomation & Threat Intelligence

MAXIMUS MUKIZA | 7/04/2026

Table of Contents

1.	Inleiding.....	7
1.1.	Probleemstelling: Trage MTTR, Vendor Lock-in en Schaalbaarheidsprobleem	7
1.2.	Projectdoelstellingen en Scope.....	8
1.3.	Onderzoeksvragen	8
1.4.	Structuur van de Bachelor Proef.....	9
2.	Theoretische kader.....	10
2.1.	Security Operations Center (SOC) en het Moderne IT-landschap	10
2.2.	Endpoint Detection and Response (EDR) vs. XDR.....	10
2.3.	Werking en Architectuur van SentinelOne Singularity.....	10
2.4.	Cyber Threat Intelligence (CTI)	11
2.4.1.	De CTI Lifecycle en Intelligence Sharing (CCB & CIRCL).....	11
2.4.2.	Indicators of Compromise (IOC), STIX en TAXII Standard.....	11
2.5.	Security Orchestration, Automation and Response (SOAR).....	12
2.5.1.	No-code/ Low-code Automation vs custom code.	12
2.5.2.	Human-in-the-loop (HITL) mechanisme in Cybersecurity	12
2.6.	Overzicht van Geëvalueerde Technologieën.....	13
2.6.1.	MISP (Malware Information Sharing Platforms)	13
2.6.2.	OpenCTI	14
2.6.3.	Hyperautomation Platforms (S1 HyperAutomate & Rewst)	15
3.	Methodologie en Projectaanpak	16
3.1.	Agile & Iteratief werken in een Security Integratie Context.....	16
3.2.	Risicoanalyse en Mitigaties strategieën	16
3.3.	Test- en validatie strategie (Unit testing, Integration, UAT).....	17
4.	Analyse en Selectie van Threat Intelligence Platform	18
4.1.	Noodzaak voor onafhankelijke Threat Intelligence	18
4.2.	Proof of Concept 1: OpenCTI.....	19

4.2.1.	Docker Deployments en Architectuur	19
4.2.2.	Knelpunten: trage ingestie, swap-geheugen en Resource verbruik	19
4.3.	Proof of Concept 2: MISP	21
4.3.1.	Integratie met CCB en CIRCL via PGP en S/MIME encryptie	21
4.3.2.	Performance Evaluatie en Keuze voor MISP	22
5.	Implementatiefase 1: Integratie van SentinelOne en MISP	23
5.1.	De eerste opzet met de SentinelOne Marketplace app	23
5.2.	Technische analyse van de fouten	24
5.2.1.	Analyse van HTTP 40000100 en 502 bad gateway fouten	24
5.2.2.	Problemen met de STIX 2.1 validatie	25
5.3.	De overstap naar eigen HTTP API verzoeken	27
5.3.1.	Gebruik van GraphQL en REST via de S1 API	27
5.3.2.	Beheer van data en paginatie	28
5.3.3.	Normalisatie naar OCSF	28
5.4.	Beheer van Docker bronnen en logging	29
6.	Implementatiefase 2: basis workflows in S1 Hyperautomation	30
6.1.	Doelstelling van de Hyperautomation Flows	30
6.2.	Ontwikkeling van de Basis Workflows	31
6.2.1.	Workflow 1: Auto Enrichment in alerts. Json	31
6.2.2.	Workflow 2: Corrupted password or credentials for email. Json	32
6.2.3.	Workflow 3: Warn client if agents are down. Json	33
6.2.4.	Workflow 4: teams alerts. Json	35
6.3.	Uitdagingen binnen S1 Hyperautomation	36
6.3.1.	Complexiteit van JSON Manipulatie met JQ	36
6.3.2.	Technische Imitaties bij Privilege Escalation en Lateral Movement	38
7.	Implementatiefase 3: Verkenning van het Rewst Platform	39
7.1.	Beperking van SentinelOne Hyperautomation	39

7.2.	Ontwerpfase en architectuur van de workflows.....	40
7.3.	Implementatie en de Datto Autotask API blokkade.....	41
7.4.	Kritische evaluatie van low-code voor SOC-operaties	42
7.4.1.	Jinja en abstractie van logica	42
7.4.2.	Snelheid en versiebeheer	44
7.4.3.	Netwerkbeveiliging.....	45
7.5.	Overstap naar python	46
8.	Implementatiefase 4: De Overstap naar python.....	47
8.1.	Beperking van NO-code platformen en de keuze voor python.....	47
8.1.1.	Versiebeheer en leesbaarheid van code	47
8.1.2.	Foutopsporing en netwerkcontrole	48
8.2.	Architectuur van python Microservices	48
8.2.1.	Polling Architectuur (standalone Scripts).....	49
8.2.2.	Event-Driven Architectuur via Webhooks en FastAPI	49
8.2.3.	Message Brokering met Rabbitmq	50
8.2.4.	Deduplicatie en State Management met Redis	51
8.3.	Implementatie van python playbooks.....	52
8.3.1.	Waarschuwingen verrijken met Threat Intelligence (Enrichment workers).....	52
8.3.2.	Script voor Gecompromitteerde Gebruikers (Credential audit)	54
8.3.3.	Controle van Agent status (Attention Monitor).....	55
8.3.4.	Tickets en notificaties Genereren (Ticketing Engine)	55
8.4.	Wijzigingen in de Operationele workflow	57
9.	Kwaliteitsborging, Testen en Validatie	58
9.1.	Opzet van de Virtuele Testinfrastructuur	58
9.1.1.	Hyper-V en Client Virtuele Machines	58
9.1.2.	Backend Server en Containerisation	58
9.2.	Simulatie van Dreigingen met EICAR en Ikarus	59

9.2.1.	Gebruik van Testbestanden	59
9.2.2.	Beperkingen bij het Testen van Gedragsdetectie	59
9.3.	Validatie van de Python Playbooks	59
9.3.1.	Unit Testing en Datavalidatie	59
9.3.2.	Integration Testing en Error Handling	60
9.3.3.	Load Testing en Redis Deduplicatie.....	60
9.4.	User Acceptance Testing (UAT).....	60
9.4.1.	Beoordeling van Ticket Kwaliteit	60
9.4.2.	Validatie van Notificaties (Microsoft Teams)	61
10.	Resultaten en Business Impact	61
10.1.	Operationele Impact: Reductie van de Mean Time to Respond (MTTR)	61
10.2.	Kwalitatieve Impact: verrijkte context en consistente Troubleshooting	62
10.3.	Architecturale Impact: Eliminatie van vendor lock-in.....	62
10.4.	Financiële Impact: Python Script vs Licentiekosten van Rewst/Hyperautomation.....	63
11.	Conclusie	63
11.1.	Beantwoording van de onderzoeksvragen	63
11.2.	Algemene conclusie.....	64
11.3.	Lessons Learned: De waarde van pivoteren in software development.....	64
12.	Toekomstvisie en Aanbevelingen	65
12.1.	Verder uitbreiding van de python playbook bibliotheek	65
12.2.	Toekomstige optimalisaties (OpenSearch/ElasticSearch integratie)	65
12.3.	Advies voor het management van VanRoey	66
13.	Literatuurlijst.....	67
14.	Bijlagen	69
14.1.	Bijlage A: Reference – API Endpoints, STIX Data Schema's JSON workflows	69
14.1.1.	SentinelOne GraphQL Endpoints.....	69
14.2.	SentinelOne Hyperautomation (Legacy JSON Workflow).....	70

14.2.1. HTTP Request configuratie.....	70
14.2.2. Gekoppelde JQ Conditions	70
14.3. Bijlage B: Reference – Python Source Code van de Microservices.....	70
14.3.1. Webhook (`soc_scripts/docker/api/webhook_receiver.py`)	70
14.3.2. Autotask Ticketing Logica (`soc_scripts/docker/workers/ticketing_engine.py`) .	71
14.4. Bijlage C: How-To Guide – SOC Runbook: Applicatiebeheer & Docker	72
14.5. Bijlage E: Explanation – Transcript en Analyse van het gefaalde SentinelOne Support Ticket	

1. INLEIDING

VanRoey is een Belgische ICT-dienstverlener die organisaties ondersteunt bij digitale transformatie, cloud-infrastructuur en cybersecurity. Binnen het bedrijf vormt het Security Operations Center (SOC) de kern van de managed Detection and Response (MDR)-dienstverlening. Het SOC monitort en beveiligt de IT-infrastructuur van verschillende klanten. Hiervoor gebruikt het team SentinelOne Singularity als primair platform voor endpoint security (EDR/XDR) en identity posture management. Dit beveiligingsplatform integreert met de bredere netwerkinfrastructuur waaronder Microsoft omgeving, Fortinet-firewalls en Operational Technology (OT) beveiligingssystemen.

1.1. Probleemstelling: Trage MTTR, Vendor Lock-in en Schaalbaarheidsprobleem

Ondanks de mogelijkheden van SentinelOne ervaart het SOC operationele knelpunten bij de afhandeling van incidenten. Analisten voeren de triage en verrijking van inkomende waarschuwingen nog grotendeels handmatig uit. Om de ernst van een mogelijke dreiging in te schatten, raadplegen zij externe databanken zoals Virustotal en AbuseIPDB, of interne lijsten. Deze handmatige werkwijze verhoogt de Mean Time To Respond (MTTR). Bij tijdgevoelige incidenten, zoals een ransomwareaanval, zorgt elke vertraging in de analyse voor grotere risico's. Daarnaast is er sprake van vendor lock-in op het vlak van Threat Intelligence (TI). De organisatie vertrouwt sterk op de ingebouwde intelligentie van de EDR-leverancier. Wanneer het platform een specifieke indicator niet herkent, wordt de dreiging mogelijk niet door het SOC opgemerkt. Het ontbreekt momenteel aan een structurele integratie met onafhankelijke, community-gedreven of sectorspecifieke bronnen (Indicator of Compromise). Tot slot leiden deze handmatige processen tot schaalbaarheidsproblemen. Met een groeiend klantenbestand neemt het aantal dagelijkse waarschuwingen toe. Om efficiënt te blijven opereren, moet incidentafhandeling worden geautomatiseerd, zodra de werkdruk van de analisten niet lineair hoeft mee te groeien met het aantal klanten.

1.2. Projectdoelstellingen en Scope

Het doel van deze bachelorproef is het automatiseren van de Threat Intelligence Ingestie en incidentrespons via SOAR-playbooks (Security Orchestration, Automation and Response).

De projectdoelstellingen zijn als volgt gedefinieerd:

- Doel 1: Automatisering van incident triage. Drie functionele workflows ontwikkelen die waarschuwingen automatisch afhandelen.
- Doel 2: Implementatie van Onafhankelijke Threat Intelligence. Een API-koppeling realiseren tussen SentinelOne en een onafhankelijk threat intelligence-platform (zoals MISP). De logica moet paginatie ondersteunen om grote hoeveelheden data efficiënt te verwerken.
- Doel 3: reductie van vendor lock-in. Een systeemarchitectuur opzetten waarbij TI-feeds onafhankelijk van de EDR-leverancier beheerd en dynamisch toegevoegd kunnen worden.

De scope van de opdracht omvat het bouwen, configureren en testen van de automatiseringslogica en API-verbindingen in een gecontroleerde testomgeving met gesimuleerde dreigingen (zoals het EICAR-testbestand). Het opzetten van de onderliggende hardware-infrastructuur voor de platformen en het schrijven van complexe log parsers van het SIEM vallen buiten de opdracht.

1.3. Onderzoeksvragen

Dit onderzoek richt zich op de volgende vragen:

- Hoe kunnen onafhankelijke Threat Intelligence-bronnen, met name MISP, technisch worden geïntegreerd met SentinelOne om de vendor lock-in op Threat data te verminderen?
- Op welke manier dragen geautomatiseerde SOAR-playbooks bij aan het efficiënter afhandelen van incidenten binnen het SOC van VanRoey?
- Welke technologie biedt de beste oplossing voor het bouwen van deze playbooks, en waarom is er vanuit een initiële verkenning van no-codeplatforms (SentinelOne Hyperautomate, Rewst) uiteindelijk gekozen voor custom Python-scripts?

1.4. Structuur van de Bachelor Proef

Deze bachelorproef is gestructureerd aan de hand van het Diátaxis-framework. Dit model scheidt theoretische uitleg en analyses (Explanation) strikt van de praktische referentiematerialen (How-To's en Reference). Hierdoor fungeert dit document niet alleen als een academisch verslag, maar ook als het handboek voor het SOC-team.

Hoofdstuk 2 vormt het theoretische kader, waarin concepten zoals EDR, CTI en SOAR worden uitgelegd. In hoofdstuk 3 worden de iteratieve methodologie en risicoanalyse beschreven. Hoofdstuk 4 behandelt de vergelijking tussen OpenCTI en MISP in de zoektocht naar een geschikt Threat Intelligence Platform.

De iteratieve fasen van de projectuitvoering volgen in de daaropvolgende hoofdstukken. Hoofdstuk 5 analyseert de falende native SentinelOne Marketplace-integratie en de technische omschakeling naar directe HTTP-API-request. Hoofdstuk 6 beschrijft de ontwikkeling van de eerste basisworkflow in SentinelOne Hyperautomation.

In hoofdstuk 7 wordt de verkenning van de rewt-platforms behandeld. Hoofdstuk 8 documenteert de definitieve overstap naar Python, waarbij wordt beargumenteerd waarom scriptgebaseerde logica de voorkeur kreeg boven visuele no-codeoplossingen. Hoofdstuk 9 beschrijft de kwaliteitsborging en User Acceptance Testing (UAT). De resultaten en de operationele impact worden geëvalueerd in hoofdstuk 10. Het document sluit af met een conclusie in hoofdstuk 11 en aanbevelingen in hoofdstuk 12.

Alle technische broncode, JQ-filters, API-referenties en beheerinstruaties zijn opgenomen in de bijlagen.

2. THEORETISCHE KADER

Binnen de context van dit project spelen verschillende technologieën en concepten een rol. Dit hoofdstuk biedt de theoretische achtergrond die nodig is om de architectuurkeuzes en technische implementaties in de latere hoofdstukken te begrijpen.

2.1. Security Operations Center (SOC) en het Moderne IT-landschap

Een Security Operations Center (SOC) is de afdeling binnen een organisatie of IT-dienstverlener die verantwoordelijk is voor het monitoren, detecteren en afhandelen van beveiligingsincidenten. Bij IT-dienstverleners zoals VanRoey levert het SOC Managed Detection and Response (MDR)-diensten aan diverse klanten. Naarmate het klantenbestand groeit, neemt de hoeveelheid dagelijkse beveiligingswaarschuwingen (alerts) proportioneel toe. Als incident triage en respons volledig handmatig worden uitgevoerd, moet het SOC-team lineair meegroeien met de waarschuwingen, wat bedrijfseconomisch onhaalbaar is. Automatisering verlaagt de werklust en verkort de Mean Time to Respond (MTTR), zodat analisten zich kunnen richten op complexe incidenten in plaats van repetitieve handelingen.

2.2. Endpoint Detection and Response (EDR) vs. XDR

Endpoint Detection and Response (EDR)-systemen monitoren eindapparaten zoals laptops en servers op verdacht gedrag en grijpen in bij dreigingen. Extended Detection and Response (XDR) breidt deze monitoring uit naar netwerk, cloud en identiteitsniveaus.

2.3. Werking en Architectuur van SentinelOne Singularity.

SentinelOne Singularity fungeert als platform voor EDR/XDR en Identity Detection & Response (IDR) binnen de organisatie. Het platform verzamelt telemetrie en normaliseert inkomende logs via het Open Cybersecurity Schema Format (OCSF) binnen de Singularity AI SIEM. Om dreigingen op te sporen in grote hoeveelheden data, maakt het systeem gebruik van Event Search voor filtering en Power Query voor formattering en datavisualisatie.

2.4. Cyber Threat Intelligence (CTI)

Cyber Threat Intelligence omvat verzamelde data over dreigingsactoren, malware en kwetsbaarheden. Uitsluitend vertrouwen op de data van een beveiligingsleverancier creëert een blinde vlek, ook wel vendor lock-in genoemd. Als de leverancier een specifieke dreiging niet herkent, detecteren de verdedigingssystemen deze niet. Integratie met onafhankelijke bronnen lost dit probleem op.

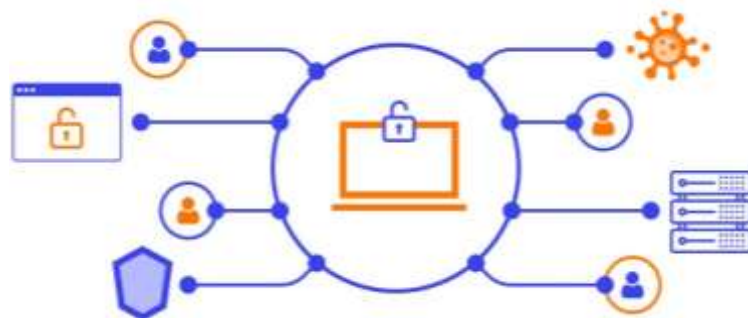
2.4.1. De CTI Lifecycle en Intelligence Sharing (CCB & CIRCL)

De CTI-levenscyclus beschrijft de stappen van dataverzameling tot en met de distributie van inlichtingen. Binnen België en Europa spelen overheidsinstanties en non-profitorganisaties een rol in het delen van deze informatie. Het Centrum voor Cybersecurity België (CCB) en het Computer Incident Response Center Luxembourg (CIRCL) bieden feeds met actuele dreigingsinformatie. Het opzetten van beveiligde communicatiekanalen via protocollen zoals S/MIME en PGP-sleutels is een vereiste om deze feeds in lokale Threat Intelligence Platforms te integreren.



2.4.2. Indicators of Compromise (IOC), STIX en TAXII Standard

Dreigingen worden geïdentificeerd aan de hand van Indicators of Compromise (IoC's), zoals IP-adressen, bestandshashes en domeinnamen. Om deze data tussen verschillende systemen te delen, worden standaarden zoals Structured Threat Information Expression (STIX) en Trusted Automated Exchange of Intelligence Information (TAXII) gebruikt. STIX definieert het formaat van de data. TAXII verzorgt het transportprotocol, waardoor platformen dreigingsinformatie via geautomatiseerde feeds (zoals TAXII2) kunnen uitwisselen.

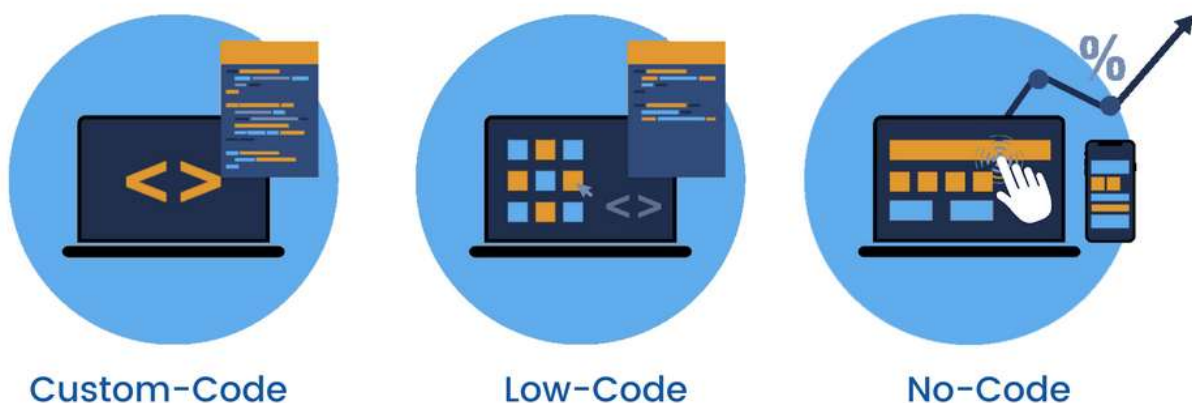


2.5. Security Orchestration, Automation and Response (SOAR)

SOAR-systemen automatiseren de processen rondom incidentrespons. Ze koppelen verschillende beveiligingstools aan elkaar en voeren vooraf gedefinieerde acties (Playbooks) uit. Dit proces start bij triage, waarbij binnengekomen alerts automatisch worden verrijkt met externe data, zoals een zoekopdracht in Virustotal of een interne Threat Intelligence-database.

2.5.1. No-code/ Low-code Automation vs custom code.

Veel moderne SOAR-platformen bieden no-code of low-code interfaces, waarbij gebruikers playbooks bouwen via visuele drag-and-drop-systemen. Platformen zoals Hyperautomation en Rewst abstraheren de achterliggende logica. Bij complexe processen vormt deze abstractie een nadeel. Het verbergen van de onderliggende datastromen bemoeilijkt het debuggen en vertraagt de workflows. Uit tests met het Rewst-platform bleek dat het configureren van workflows (zoals scenario's voor gebruikers met slechte wachtwoorden) onnodig gecompliceerd werd door de interface. Omdat Python breed begrepen wordt binnen het SOC-team, resulteert het schrijven van Python-scripts in transparantere, snellere en gemakkelijker te onderhouden processen ten opzichte van low-code-abstracties.



2.5.2. Human-in-the-loop (HITL) mechanisms in Cybersecurity

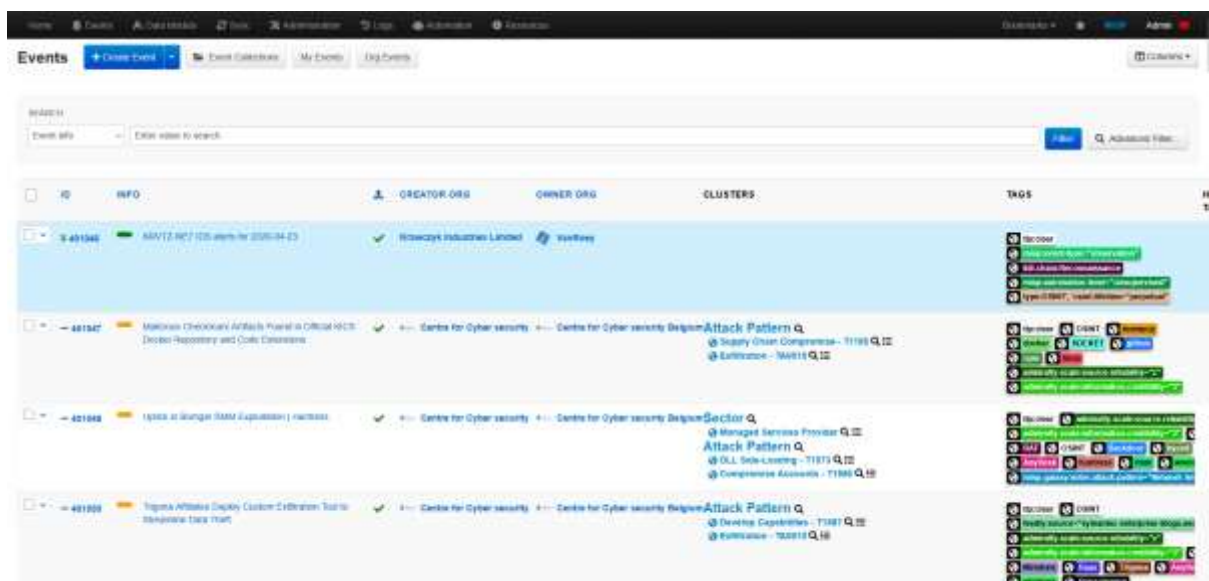
Volledige automatisering brengt het risico op foutieve beslissingen met zich mee. Een false positief kan leiden tot het onterecht isoleren van een server, wat de bedrijfsvoering verstoort. Een human-in-the-loop (HITL)-architectuur vereist dat systemen zelfstandig data verzamelen en verrijken, waarna ze mitigatieadvies klaarzetten. Een analist geeft vervolgens handmatige goedkeuring voordat een destructieve actie wordt uitgevoerd.

2.6. Overzicht van Geëvalueerde Technologieën

Tijdens het project zijn verschillende platformen getest voor de Threat Intelligence-integratie.

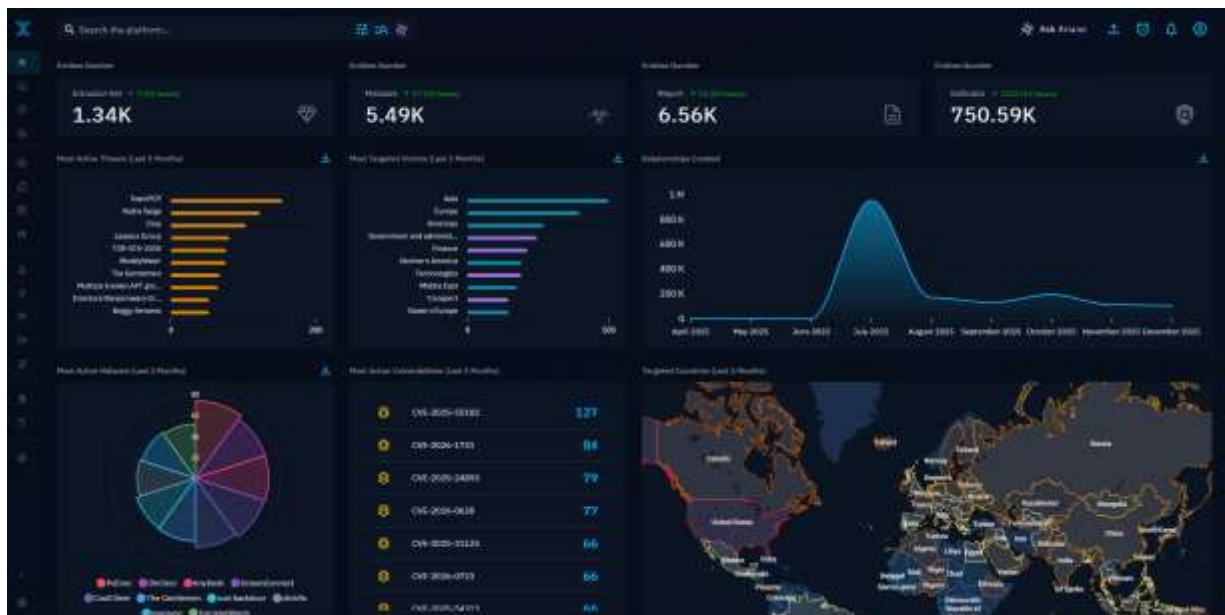
2.6.1. MISP (Malware Information Sharing Platforms)

MISP is een open-sourceplatform voor het delen en opslaan van dreigingsinformatie. Tijdens de evaluatiefase bleek, in vergelijking met andere systemen, de gebruikersinterface van MISP gedateerd. Het platform beschikt echter over efficiëntere automatiseringsmogelijkheden en databasestructuren. MISP verwerkt stabiel grote hoeveelheden IoC's API's en feeds en is daarom gekozen als het primaire platform.



2.6.2. OpenCTI

OpenCTI is een platform voor de opslag en analyse van cyberdreigingen. Tijdens de testfase is OpenCTI via Docker geïnstalleerd. Het platform heeft een modernere grafische interface en ondersteunt standaarden zoals STIX en TAXII2. De praktijktests wezen uit dat de data-ingestie bij OpenCTI relatief traag verliep en gepaard ging met een hoog resourceverbruik, wat leidde tot swapgeheugentekorten op de server. Vanwege deze prestatieproblemen is OpenCTI niet verder meegenomen in de integratie.



2.6.3. Hyperautomation Platforms (S1 HyperAutomate & Rewst)

S1 HyperAutomate is de SOAR-module van SentinelOne. Tijdens het project werd de integratie met MISP initieel via de ingebouwde SentinelOne Marketplace-apps en HyperAutomate getest. De native integratie vertoonde aanhoudende backend-fouten (HTTP 400 en 502) bij de verwerking van IOC-patronen doordat de achterliggende parsing van SentinelOne vastliep. Rewst is een extern low-code IT-automatiseringsplatform dat daarnaast werd geëvalueerd voor het bouwen van de playbooks. Beide platformen bleken door hun visuele abstractielagen te complex en omslachtig voor de behoeften van het SOC. Het project is daarna overgestapt op maatwerk via API-request en Python-scripts



3. METHODOLOGIE EN PROJECTAANPAK

3.1. Agile & Iteratief werken in een Security Integratie Context

Tijdens de uitvoering wordt een Agile-geïnspireerde methodiek met iteraties van twee tot drie weken gebruikt. De integratie van externe API's in een cybersecurityomgeving brengt onzekerheden met zich mee rond verbindingen en parsing. Een sequentiële watervalmethode is hierbij onpraktisch omdat technische limieten pas zichtbaar worden tijdens het testen van de API-endpoints.

Er is gewerkt met een backlog voor de te ontwikkelen functionaliteiten. Per sprint is bepaald welke taken prioriteit kregen. In de tweede sprint (week 4 tot en met 6) zijn direct de koppelingen met MISP en de SentinelOne API gebouwd. Hierdoor werd snel duidelijk dat de native SentinelOne-connector niet werkte wegens backend-parsingfouten. Door deze iteratieve aanpak was er de mogelijkheid om vroeg in het project over te schakelen naar rechtstreekse HTTP-requests en in een latere fase naar Python-scripts.

3.2. Risicoanalyse en Mitigaties strategieën

Bij de aanvang van het project is een risicoanalyse gemaakt. Een van de operationele risico's was 'scope creep', oftewel het gaandeweg toevoegen van extra wensen vanuit het soc. Dit is opgevangen door een strak afgebakende scope in het projectplan vast te leggen.

Daarnaast was er het technische risico op API Rate Limiting door externe diensten. Een ander technisch risico betrof het ophalen van bulkdata uit MISP. Het downloaden van duizenden indicatoren tegelijk veroorzaakt time-outs of limietoverschrijdingen aan de serverkant. Om dit op te lossen is in de code gebruikgemaakt van paginatie met cursors, waarbij data in afzonderlijke batches wordt opgevraagd en weggeschreven.

Wat betreft operationele veiligheid was er het risico op false positives, waarbij een geautomatiseerde actie ten onrechte een kritieke server van de klant isoleert. Dit is gemitigeerd door integratie van het human-in-the-loop (HITL)-mechanisme in de flow, wat betekent dat een analist altijd handmatig goedkeuring geeft voor ingrijpende acties. Het privacy risico rondom klantinformatie is ondervangen in het architectuurontwerp door enkel bestandshashes of losse IP-adressen zonder bedrijfcontext te verwerken.

3.3. Test- en validatie strategie (Unit testing, Integration, UAT)

Het testen van de API's en scripts gebeurt in drie afzonderlijke fasen.

Voor de unit tests is Postman gebruikt. Hiermee werd op API-niveau getest voordat enige code in de playbooks werd geïmplementeerd. Er is gekeken naar de benodigde authenticatie-header en de structuur van de JSON-responses van de verschillende endpoints.

De integratietesten vonden plaats op het niveau van de workflow. Hierbij werd de foutafhandeling gecontroleerd. Er is getest hoe het script reageert op een foutmelding (zoals een '404 Not Found' of lege velden in een payload) en of de uitvoering netjes wordt afgevangen zonder de rest van het proces te laten vastlopen.

Voor de User Acceptance Testing (UAT) is een virtuele testinfrastructuur opgezet met Windows- en Linux-clients op Hyper-V. Samen met de SOC-analisten werden dreigingen gesimuleerd door de executie van EICAR-testbestanden op de endpoints. Tijdens deze tests werd geobserveerd of de SentinelOne-agent de waarschuwing registreerde en of de Python-scripts vervolgens correct werden uitgevoerd om de data bij de analist af te leveren.



4. ANALYSE EN SELECTIE VAN THREAT INTELLIGENCE PLATFORM

Dit hoofdstuk beschrijft de initiële onderzoeks- en testfase (uitgevoerd in de eerste twee weken van het project) waarin twee prominente Threat Intelligence Platforms (TIP's) tegen elkaar werden afgewogen: OpenCTI en MISP (Malware Information Sharing Platform). Het doel van deze fase was om een schaalbaar, onafhankelijk platform te selecteren dat naadloos kon integreren met SentinelOne en voldeed aan de eisen van het Security Operations Center (SOC) van VanRoey.

4.1. Noodzaak voor onafhankelijke Threat Intelligence

Voordat de technische evaluatie van de platformen begon, moest de strategische noodzaak voor een extern TIP worden vastgesteld. Binnen de initiële situatie vertrouwde het SOC van VanRoey sterk op de ingebouwde Threat Intelligence van de primaire EDR-leverancier (SentinelOne). Hoewel dit een solide basis laag aan beveiliging beidt, creëert deze architectuur twee significante knelpunten:

1. Vendor Lock-in en blinde vlekken: wanneer een organisatie uitsluitend vertrouwt op de intelligentie van een leverancier, ontstaat er een blinde vlek. Als de leverancier een specifieke threat actor, lokaal georiënteerde malware of een zero-day-kwetsbaarheid niet (tijdig) herkent, zal het SOC ook geen detectie op hebben.
2. Gebrek aan community-Gedreven context: Het ontbreken van een integratie met onafhankelijke, sectorspecifieke of nationale bronnen (zoals het Centrum voor Cybersecurity België - CCB) betekent dat waardevolle Indicators of Compromise (IOC's) niet automatisch worden meegenomen in de incident triage.

Door een onafhankelijk Threat Intelligence Platform te introduceren, ontkoppelt de organisatie haar intelligentie van de detectie-engine. Dit stelt het SOC in staat om dynamische dreigingsinformatie toe te voegen, externe feeds te correleren en de Mean Time To Respond (MTTR) te verlagen door analisten direct te voorzien van verrijkte context uit meerdere bronnen.

4.2. Proof of Concept 1: OpenCTI

De eerste kandidaat in het evaluatieproces was OpenCTI, een modern open-sourceplatform dat bekend staat om zijn uitgebreide grafische weergaven en sterke ondersteuning voor STIX 2.1-standaarden. Tijdens de eerste projectweek werd OpenCTI succesvol uitgeroeid om de functionaliteiten in de praktijk te toetsen.

4.2.1. Docker Deployments en Architectuur

De initiële deployments van OpenCTI werden lokaal opgezet via Docker Compose. Al snel bleek dat de architectuur van OpenCTI (die sterk leunt op meerdere microservices zoals Elasticsearch/OpenSearch, Redis, Rabbitmq en diverse Python workers) aanzienlijke hardware vereisten had. De lokale ontwikkel-laptop beschikte over onvoldoende rekenkracht om de volledige stack stabiel te draaien. Om een representatieve testomgeving te garanderen, werd er overgeschakeld naar een dedicated Virtual Machine op een VMware ESXi-server.

Op deze server werd OpenCTI verder geconfigureerd, inclusief de toevoeging van specifieke TAXII2-connectoren en plug-ins om te beoordelen hoe het platform data importeerde. De user interface (UI) bleek zeer intuïtief en de visuele weergave van complexe dreigingslandschappen was superieur. Echter, bij het testen van de geautomatiseerde workflow kwamen de eerste tekortkomingen aan het licht.

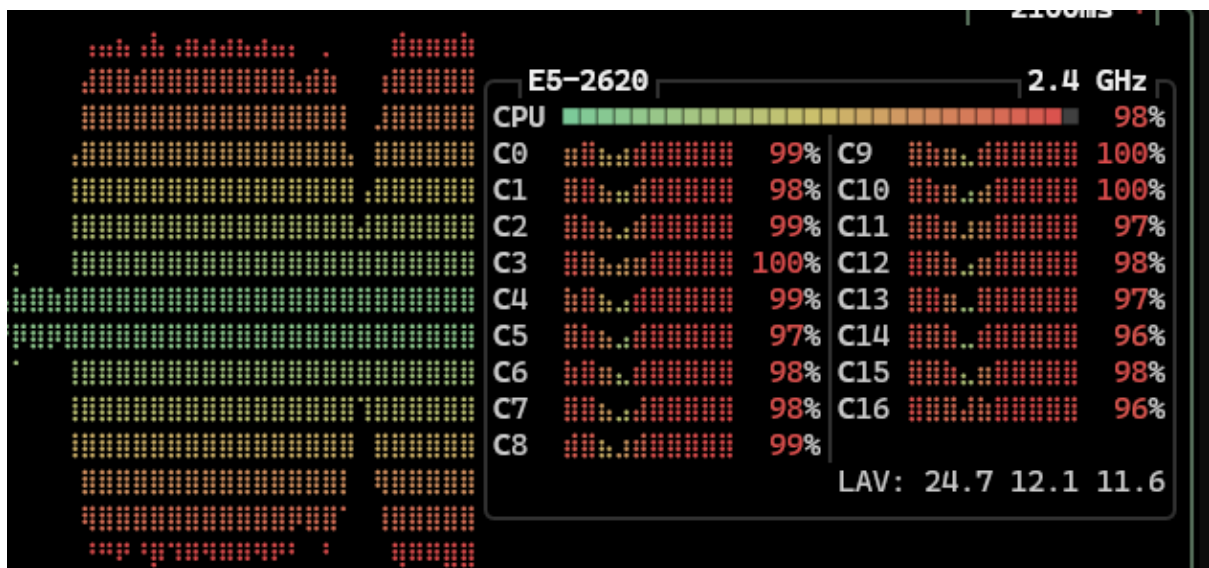
4.2.2. Knelpunten: trage ingestie, swap-geheugen en Resource verbruik

Tijdens het simuleren van real-world data-ingestie in week 2, waarbij grote hoeveelheden Threat Intelligence werden ingeladen, liep de infrastructuur tegen harde limieten aan. De synchronisatie liep vast, waarbij de data-ingestie bleef steken op historische data uit 2021.

Uit verder technische analyse bleek dat het werd veroorzaakt door een tekort aan swapgeheugen op de server. Het zware resourceverbruik van de onderliggende Elasticsearch-nodes met de naam blokkeerde de ingestiepijplijn. Om dit te mitigeren werden verschillende optimalisatie doorgevoerd:

- De server werd geherconfigureerd om prioriteit te geven aan dataingestietaken.
- Het aantal actieve 'workers' werd verdubbeld in een poging om data sneller te verwerken
- Er werd onderzoek gedaan naar lichtere alternatieven voor de database laag, zoals OpenSearch, Algolia en Typesense.

Hoewel deze optimalisaties het proces tijdelijk herstelden, toonde de Proof of Concept aan dat OpenCTI voor de specifieke, high-volume automatiseringsbehoefte van dit project te zwaar en te complex was om betrouwbaar en resource-efficiënt te draaien zonder significante infrastructuurinvesteringen.



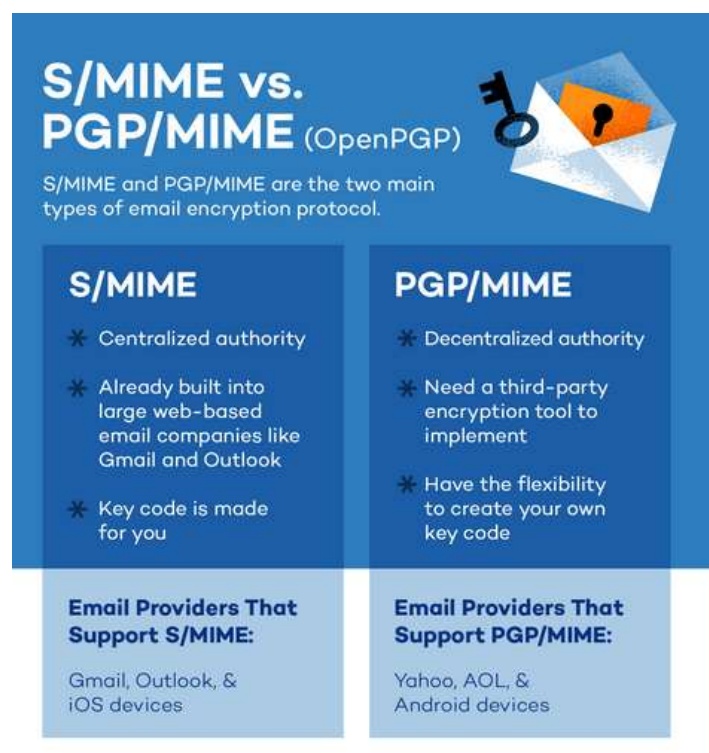
4.3. Proof of Concept 2: MISP

Parallel aan OpenCTI werd het Malware Information Sharing Platform (MISP) geëvalueerd. Waar OpenCTI uitblonk in visualisatie, bleek MISP direct een voorsprong te hebben op het gebied van pure automatisering en naadloze data-uitwisseling tussen beveiligingsorganisaties.

4.3.1. Integratie met CCB en CIRCL via PGP en S/MIME encryptie

Een van de absolute randvoorwaarden voor het nieuwe TIP was het succesvol kunnen aansluiten op gerenommeerde intelligentiebronnen, waaronder het CCB en CIRCL (Computer Incident Response Center Luxembourg).

Tijdens het opzetten van deze verbindingen ontstond een infrastructurele uitdaging: de standaardcommunicatie met deze instanties vereiste beveiligde verbindingen, vaak via S/MIME. Aangezien S/MIME op dat moment niet direct beschikbaar was binnen de omgeving van het bedrijf, moest er een alternatieve cartografische route worden opgezet. Er werden succesvolle PGP/GPG-sleutels gegenereerd en uitgewisseld. Na het configureren van deze encryptiestandaard kon er succesvol toegang worden verkregen tot de CIRCL MISP-server. De feed werd toegevoegd aan de lokale MISP-instantie en de synchronisatie van actuele, hoogwaardige Threat Intelligence-data verliep zonder problemen.



4.3.2. Performance Evaluatie en Keuze voor MISP

Om de sterke punten van beide platformen te combineren, werd in eerste instantie geprobeerd om MISP (als efficiënt data-aggregaten) te koppelen aan OpenCTI (voor de visuele weergave en analyse) via een ingebouwde connector. Deze setup bleek echter onnodig veel overhead en synchronisatieproblemen op te leveren.

Bij een direct vergelijking kwamen de volgende conclusies naar voren:

- Gebruikersinterface: MISP heeft een steilere leercurve en een minder moderne interface vergeleken met OpenCTI
- Automatisering en Performance: MISP is aanzienlijk lichter in resourceverbruik, robuuster in het verwerken van bulkdata en beschikt over superieure API-gedreven automatiseringsmogelijkheden.

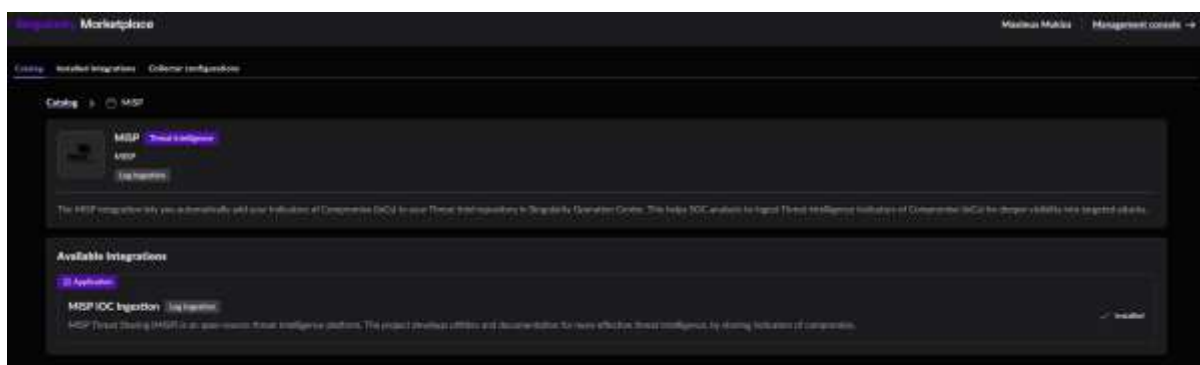
Gezien de primaire doelstelling van het project het automatiseren van incident triage (SOAR) betrof en niet zozeer het handmatige en visuele analyseren van dreigingsactoren door analisten, viel de uiteindelijke architecturale beslissing in het voordeel van MISP. De MISP-instantie werd vervolgens gestabiliseerd, voorzien van extra opslag, en achter een lokaal domein geplaatst (met de vereiste HTTPS-configuratie) ter voorbereiding op de kritieke integratiefase met SentinelOne in week 3.

5. IMPLEMENTATIEFASE 1: INTEGRATIE VAN SENTINELONE EN MISP

De eerste fase van de bouw was gericht op het verrijken van gegevens. In een Security Operations Center (SOC) bepaalt de snelheid van informatie hoe snel een team kan reageren op een dreiging. We wilden het Malware Information Sharing Platform (MISP) koppelen aan SentinelOne (S1). Dit moest de tijd voor handmatige triage inkorten en de Mean Time to Respond (MTTR) verlagen. Het plan was om indicatoren (IoC's), zoals bestandshashes en IP-adressen, te vergelijken met lijsten van instanties zoals het CCB en CIRCL. Tijdens deze fase bleek dat de standaardoplossingen van leveranciers vaak niet werken in omgevingen met veel data. De afhankelijkheid van gesloten software bleek een technisch risico voor het SOC.

5.1. De eerste opzet met de SentinelOne Marketplace app

De eerste poging begon met de SentinelOne Marketplace. Dit is een portaal met modules die API-koppelingen automatiseren. De bedoeling is dat een team hiermee koppelingen maakt zonder zelf code te schrijven. De MISP-app in dit portaal is een integrator die een verbinding legt tussen de EDR-console en de dreigingsdatabase. Volgens de documentatie haalt deze module metadata uit alerts en zoekt deze op in een MISP-server. Als er een match is, schrijft de module de context terug naar het incident in de SentinelOne-interface. Dit gaat om labels, dreigingsniveaus en links naar MISP-events.



Voor de inrichting van deze module waren de MISP-URL en een API-sleutel (Auth-key) nodig. De testopstelling bestond uit een lokale MISP-server in een Docker-omgeving op een Ubuntu-virtuele machine. Deze server had 16 GB RAM en 4 CPU-cores. In de configuratie van het Docker Compose-bestand gebruikten we de standaardinstellingen voor de MariaDB-container. De verbinding liep via poort 80 (HTTP). We gingen ervan uit dat de cloud van SentinelOne verbinding kon maken met het publieke IP-adres van deze server via een REST-aanroep. De verwachting was dat de hash van een verdacht bestand direct gecontroleerd zou worden tegen de feeds. Na de installatie bleek echter dat er geen data werd uitgewisseld. De alertnoties in de SentinelOne-interface bleven leeg, ook al zaten de indicatoren in de gekoppelde MISP-feeds. De interface gaf geen foutmelding. Dit was de aanleiding voor een onderzoek in de backend.

5.2. Technische analyse van de fouten

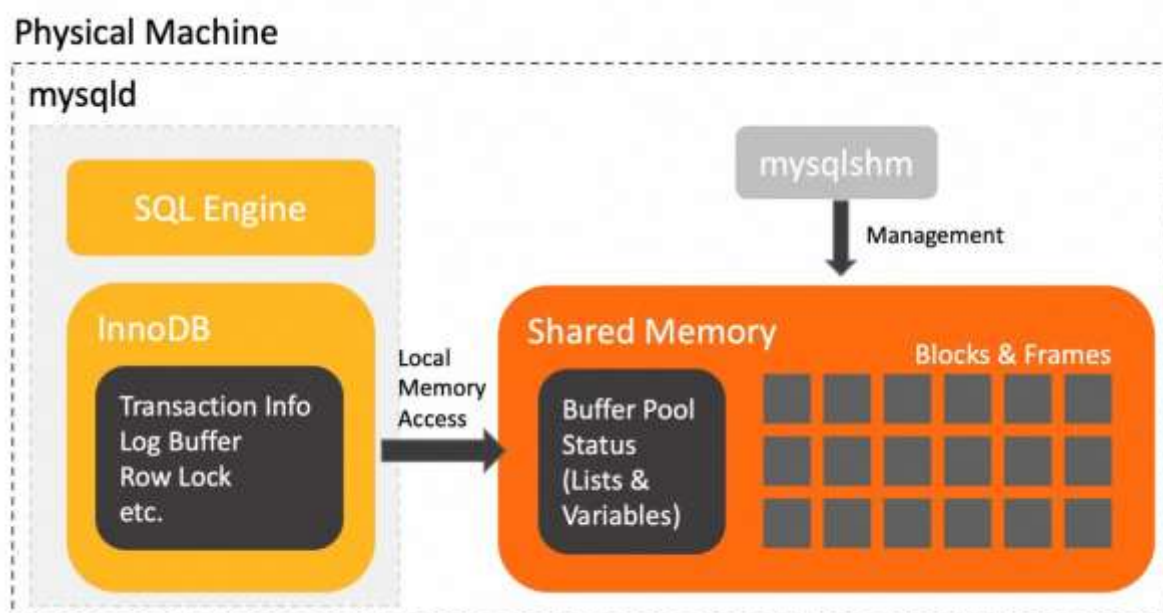
Omdat de interface geen informatie gaf, keken we in de logs van systemen. We gebruiken de SentinelOne API-endpoints voor auditing om de status van de uitgaande verzoeken te zien. Hieruit bleek dat de Marketplace-app fouten maakte bij het protocol en de interpretatie van de data. We hebben de API-endpoints handmatig getest met Postman en de resultaten vergeleken met de app.

5.2.1. Analyse van HTTP 40000100 en 502 bad gateway fouten

In de logs vonden we foutcode 4000010 terug. Volgens de documentatie van de SentinelOne API-gateway duidt dit op een “Invalid request”. We bekeken het netwerkverkeer via een TCP-netwerkdump en stelden vast dat de oorzaak lag bij het beveiligingsbeleid van de cloud-infrastructuur van SentinelOne. SentinelOne vereist namelijk HTTPS voor alle communicatie met externe integraties om de integriteit van de data te beschermen. Omdat de lokale MISP-server enkel bereikbaar was via HTTP, brak de gateway van SentinelOne de verbinding al af voordat de applicatie werd bereikt. Er bleek geen uitzonderingen op dit beleid mogelijk te zijn.

Om dit op te lossen, moesten we de netwerktoegang aanpassen. We voorzagen de MISP-server van een domeinnaam en configureerden een HA-Proxy-instantie in een aparte Docker-container die fungeerde als reverse proxy. In de configuratie van HA-Proxy werd een front-end op poort 443 opgezet voor het SSL-certificaat, terwijl de backend het verkeer doorstuurde naar poort 80. Daarnaast stelden we Access Control Lists (ACL's) in, zodat uitsluitend de IP-adressen van de SentinelOne-cloud toegang kregen. Na de implementatie van een SSL-certificaat van Let's Encrypt was de foutcode 4000010 verholpen.

Kort daarna trad een nieuwe fout op: 502 Bad Gateway. Dit bleek het gevolg van een timeout. De Marketplace-app zocht via het /attributes/restSearch-endpoint van MISP, maar de onderliggende database bevat honderdduizenden regels door de feeds van CIRCL en het CCB. Hierdoor moest de MariaDB-database bij elke zoekopdracht een SQL-query uitvoeren op een dataset die groter was dan het beschikbare geheugen. In de database-logs zagen we bovendien “lock wait timeout exceeded”-foutmeldingen, die verklaard konden worden door een te lage waarde voor de innodb_buffer_pool_size. De reactietijd van de MISP-server liep hierdoor vaak op tot boven de 40 seconden, terwijl de proxy van SentinelOne de verbinding na 30 seconden automatisch verbreekt. Dit veroorzaakte de 502-fout; de Marketplace-app bleek technisch niet in staat om te gaan met de latentie van een lokale database.



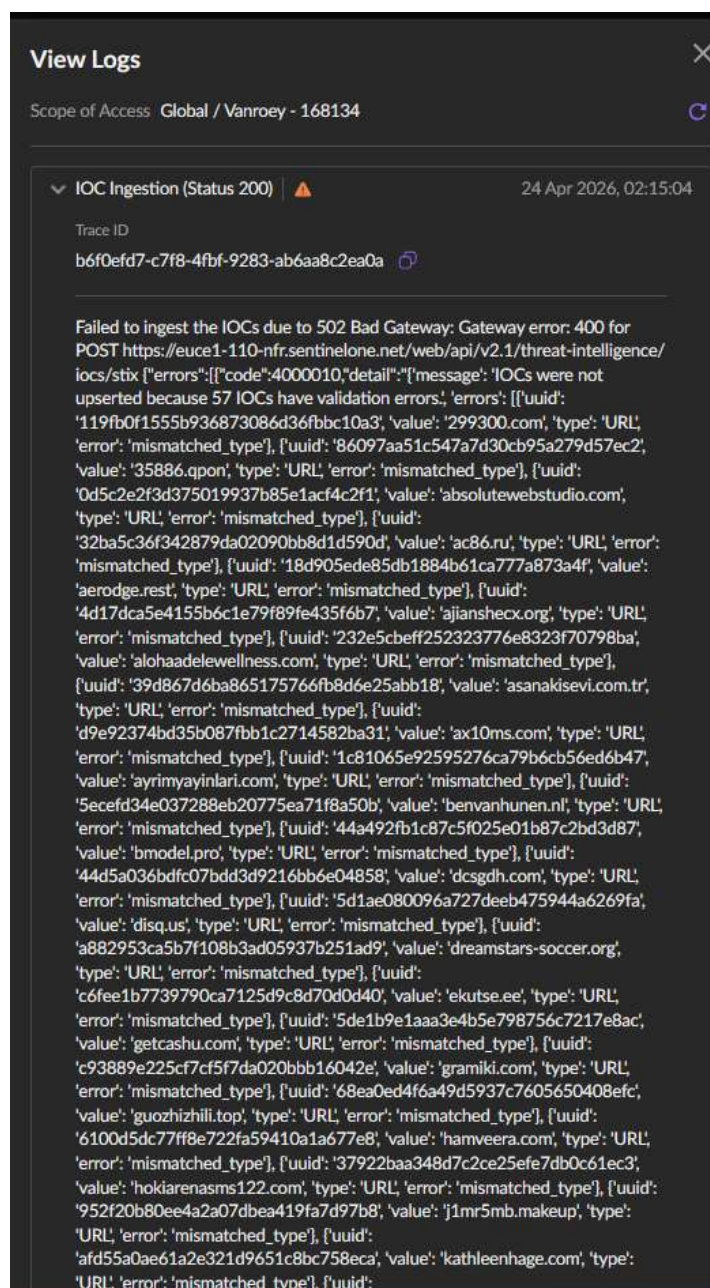
5.2.2. Problemen met de STIX 2.1 validatie

Er waren ook problemen met het verwerken van de JSON-data. MISP gebruikt STIX (Structured Threat Information Expression) voor het uitwisselen van gegevens. De feeds van CIRCL gebruiken versie 2.1. In STIX 2.1 zijn objecten zoals malware-indicatoren direct gekoppeld aan relaties tussen verschillende actoren. Een JSON-bericht kan een “indicator” bevatten die via een “relationship” verbonden is met een “malware”-object. Dit object is dan weer verbonden met een “Vulnerability” (CVE-ID).

In de logs zagen we dat de parser van de S1 Marketplace-app deze STIX 2.1-objecten niet begreep. De software was geschreven voor een oude logica die alleen een platte lijst van indicatoren verwachtte. Als je JSON-data diepe nesting of lijsten van objecten bevatte,

werkte de validatie van de parser niet. De app stopte met het verwerken van de data zonder een foutmelding of resultaat te geven.

Dit probleem is gemeld bij de support van SentinelOne, waarna een onderzoek van enkele weken volgde. Op basis van de door ons aangeleverde API-dumps en HAR-bestanden bevestigde hun R&D-team dat de MISP-app een fout bevatte in de parser voor geneste STIX-data. Omdat er geen termijn voor een oplossing kon worden toegezegd, bleek de integratie technisch niet betrouwbaar genoeg. Om de voortgang van het project niet verder te belemmeren door de afhankelijkheid van deze defecte software, hebben we besloten de Marketplace-app niet langer te gebruiken.



Cc: Jose Soares <joseotaviof@sentinelone.com>; Maximus Mukiza <maximus.mukiza@vanroey.be>
 Subject: Re: SentinelOne Partner Newsletter – Happy AI-ster! 🎉

Hi Nikolai,

good news, I did further research and can confirm that we have already identified this issue as a bug that we are actively working on, and we will release a fix by the end of the month.

Grazie,

Stefano Morotti
 Solutions Engineer

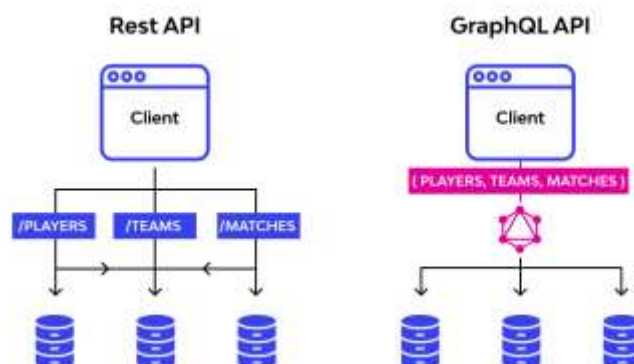
5.3. De overstap naar eigen HTTP API verzoeken

Het mislukken van de officiële app leidde tot een nieuwe aanpak. We sloten om een eigen integratie te maken met de Hyperautomation-engine van SentinelOne. Hierdoor hadden we zelf de controle over de opbouw van de verzoeken en de filtering van de data uit MISP.

5.3.1. Gebruik van GraphQL en REST via de S1 API

De nieuwe opzet gebruikte handmatige API-aanroepen in Hyperautomation-Nodes. De workflow reageerde op de interne Response Trigger van SentinelOne. Hierdoor konden we direct bij een detectie de data opvangen. We gebruikten geen polling-mechanisme meer.

We hebben de Rest-API en de GraphQL-interface van SentinelOne vergeleken. De REST-API geeft voor elke alert een JSON-bestand van ongeveer 40 KB. Dit bevat veel data over processen en apparaten die niet nodig zijn voor een check in MISP. Het verwerken van deze data kostte de werkers te veel tijd. We kozen daarom voor GraphQL. Door het GraphQL-schema te bekijken met `_schema` query's vonden we de veldnamen voor de metadata van alerts. Met een query vroegen we alleen de SHA1-hash, de hostnaam en het groep-ID op. De payload was hierdoor 85 punten kleiner. De workflow werkte sneller en er waren geen time-outs meer. We konden ook eigen headers en tokens toevoegen, wat de verbindingfouten oploste.



5.3.2. Beheer van data en paginatie

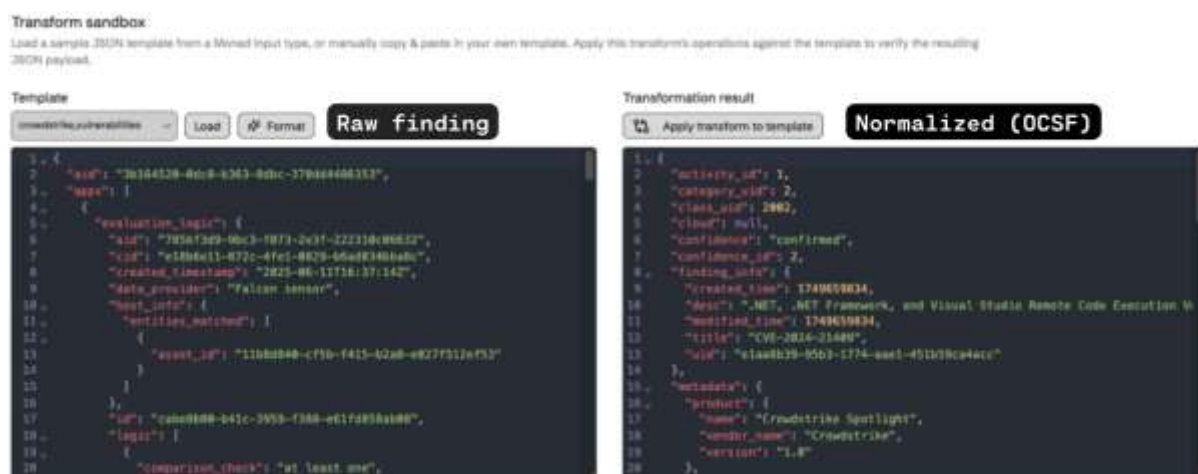
De verwerking van veel resultaten was een probleem bij de Marketplace-app. Als een indicator in veel dreigingscampagnes voorkomt, geeft MISP veel matches terug. Een grote API-aanroep zonder limiet zorgt voor een te grote response-body en time-outs.

We hebben daarom paginatie ingebouwd in de Hyperautomation-workflow. We gebruikten de parameters limit en offset in de MISP-API. De workflow haalt data op in batches van 50 records. In de workflow-engine maakten we een loop met conditionele nodes. De workflow kijkt na elke batch of de total_count uit het antwoord groter is dan het aantal opgehaalde records. Als dat zo is, volgt er een nieuwe aanroep met een hogere offset. De resultaten komen in een lijst en gaan daarna naar de volgende stap in de verwerking.

5.3.3. Normalisatie naar OCSF

Een ander onderdeel was de normalisatie van de data volgens het Open Cybersecurity Schema Format (OCSF). SentinelOne probeert dat te normaliseren, maar bij de MISP-data waren deze niet volledig. De Marketplace-app vertaalde de dreigingsniveaus van MISP niet naar de juiste OCSF-velden in de SentinelOne SIEM.

In de eigen workflow hebben we dit opgelost met JQ-scripts. We maakten een tabel waarin de MISP threat_level_id (bijvoorbeeld 1 voor high) werd gekoppeld aan de OCSF severity_id. Hierdoor kregen de verrijkte alerts de juiste categorie in de SIEM. Andere regels en het dashboard in het platform konden deze data direct gebruiken voor analyse.



5.4. Beheer van Docker bronnen en logging

Tijdens het debuggen waren er problemen met de stabiliteit van de MISP-server. De Docker-containers stopten soms door een tekort aan schijfruimte en een hoge CPU-belasting. Uit de systemlogs bleek dat de logs van systemd en de debug-logs van MISP de schijf vulden. Dit kwam door het vele API-verkeer tijdens de tests.

We hebben dit opgelost door limieten in te stellen in de ``docker-compose.yml`` file met `mem_limit` en `CPU's`. Ook hebben we logrotatie ingesteld voor de containers. De configuratie van MariaDB werd aangepast om de `innodb_buffer_pool_size` te verhogen naar 4 GB. Dit was nodig voor de SQL-Queries die MISP gebruikt bij `restSearch`-acties. Deze aanpassingen zorgden voor een stabiele server.

De ervaring met de falende Marketplace was de belangrijkste les in deze fase. Het toonde aan dat automatisering in een SOC kennis vereist van protocollen, datastructuren en databases. Deze stap was de basis voor de latere keuze om over te stappen op Python-scripts. Hiermee was de performance hoger en waren er geen beperkingen van een grafische interface. De problemen met de software van de leverancier boden aan om zelf oplossingen te ontwikkelen.

6. IMPLEMENTATIE FASE 2: BASIS

WORKFLOWS IN S1 HYPERAUTOMATION

Dit hoofdstuk beschrijft de tweede fase van de implementatie in de weken 4 en 5. De eerste fase bekeek externe TI-platforms zoals MISP en de koppeling met SentinelOne. Deze fase gaat over de automatisering binnen het SentinelOne Singularity-platform zelf. De opzet was om SOAR-theorie te gebruiken voor werkende playbooks in de Hyperautomation-module. De tekst analyseert hoe de eerste vier workflows zijn in leven gebracht. De focus ligt op de doelen, de bouw en de problemen in de low-codeomgeving. Er is aandacht voor JSON-verwerking met JQ en de moeite die het platform had met het nabootsen van aanvallen zoals Privilege Escalation. Het platform had beperkingen bij het bouwen van complexe scenario's voor het SOC-team van VanRoey.

6.1. Doelstelling van de Hyperautomation Flows

De workflows in Hyperautomation moesten de Mean Time to Respond (MTTR) van het SOC bij VanRoey verlagen. In hoofdstuk 1 staat dat de oude methode veel handwerk vroeg. Analisten moesten bij een alert zelf data zoeken. Ze kopieerden hashes en IP-adressen naar sites als Virustotal, AbuseIPDB of het eigen TI-platform. Dit werk kostte tijd en leidde soms tot fouten, zeker buiten kantoor tijd. Het team koos voor SentinelOne Hyperautomation om dit op te vangen. Omdat dit in de Singularity-omgeving zit, leek de koppeling met EDR-data makkelijker zonder extra software.

De doelen waren

1. Automatisering van Incident Triage: Playbooks bouwen die bij alerts zelf data uit systemen halen en aanvullen
2. Standaardisatie van communicatie: vaste kanalen inrichten via Teams voor analisten en e-mail voor klanten
3. Integratie met het Ticketing Systeem: De basis leggen voor een koppeling met Datto Autotask voor tickets met klant- en dreigingsdata.
4. Human-in-the-loop: zorgen dat zware acties zoals host-isolatie toestemming vragen aan een ervaren analist om storingen te voorkomen.

In week 4 en 5 werden deze doelen vertaald naar vier workflows. De werkwijze veranderde door input van analisten zoals Matthias, Joey en Milan. Ze lieten zien welke data nodig

waren en wat ze extra zouden willen zien voor hun werk. Hun kennis zorgde voor een betere weergave van info in de dagelijkse praktijk.

6.2. Ontwikkeling van de Basis Workflows

De bouw van de workflows vroeg om kennis van de SentinelOne API, webhook en dataverwerking. Ieder playbook kreeg een trigger als startpunt en een lijst met acties voor de volgende stap. In de tekst hieronder leest u een technische uitleg van de vier workflows die zijn gemaakt in de weken 4 en 5.

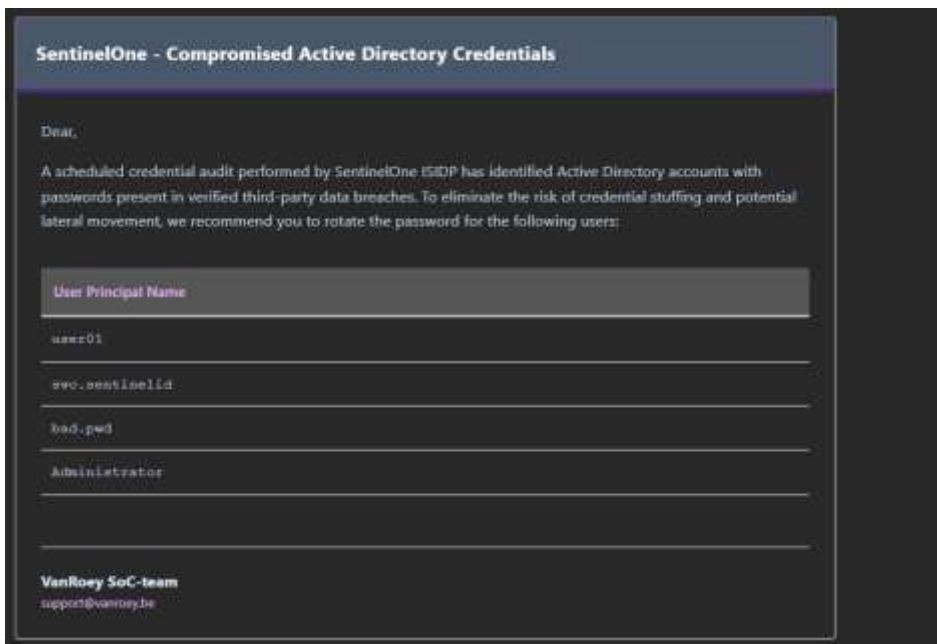
6.2.1. Workflow 1: *Auto Enrichment in alerts. Json*

De `Auto Enrichment` workflow was de eerste stap in het proces. Deze flow regelt het grootste deel van de automatische triage. Wanneer SentinelOne een dreiging ziet, zoals een vreemd bestand of een foute verbinding, maakt het systeem een alert aan in JSON-formaat. De flow start door een trigger die reageert op nieuwe alerts. De workflow haalt dan Indicators of Compromise (IOC's) uit de JSON-data. Dit zijn vaak SHA256 of SHA1 hashes, IP-adressen van externe verbindingen of domeinnamen van command-and-control servers. Daarna doet het script via API-aanroepen zoekopdrachten in bronnen zoals MISP of OTX. Omdat de standaard MISP-koppeling niet werkte, was dit script nodig om data zelf op te halen via een REST API-connector die we zelf hebben geconfigureerd. De info van MISP laat zien of een hash bekend is bij instanties als het CCB of CIRCL en of deze gekoppeld is aan specifieke malwarefamilies of campagnes van bekende threat actors.

De workflow verwerkt deze data vervolgens automatisch door de JSON-velden toe te voegen aan de analyst notes. Op advies van collega's Matthias en Joey op VanRoey is de output van dit script verbeterd. De verrijkte data komt nu direct als analyst notes in het SentinelOne Incident te staan. Dit bevat: een link naar Virustotal voor controle, een referentie naar de console, Autotask-ticket. Dit script bracht de tijd voor de eerste analyse terug van minuten naar seconden. De analist ziet bij het openen van de melding meteen alle externe info die nodig is voor de triage zonder handmatig te hoeven schakelen tussen verschillende tabbladen of portals.

Uit gesprekken met het team bleek dat incidenten rond gelekte inloggegevens veel tijd vroegen. De `CPCE` Flow is gemaakt om dit proces te verbeteren binnen de IDR-functies van SentinelOne. Het script start wanneer SentinelOne gebruikerswachtwoorden terugvindt in de Windows Active Directory-database die ook op lijsten staan met gelekte wachtwoorden die SentinelOne bijhoudt. Een lastig punt was de automatisering van de e-mails. Het doel was om klanten vanzelf te informeren over een lek met een HTML-bericht. Binnen Hyperautomation moest de HTML-code zelf worden opgebouwd met data uit de alert.

Dit vroeg om JSON-bewerkingen met JQ. De scripts moesten velden zoals gebruikersnaam en de host ophalen en in de HTML-tekst plaatsen zodat de mail goed aankwam bij de klant. Naast de mail past deze flow ook de status van de alert aan naar `In progress`. Het laat notities achter over een snellere reactie op de klant, wat de algemene kwaliteit van de dienstverlening verbetert en de kans op verdere schade door het lek verkleint. Dit script is een waardevolle toevoeging aan het arsenaal van het SOC-team van VanRoey en helpt bij de dagelijkse taken van de security experts die de omgeving bewaken tegen indringers en datalekken op grote schaal.

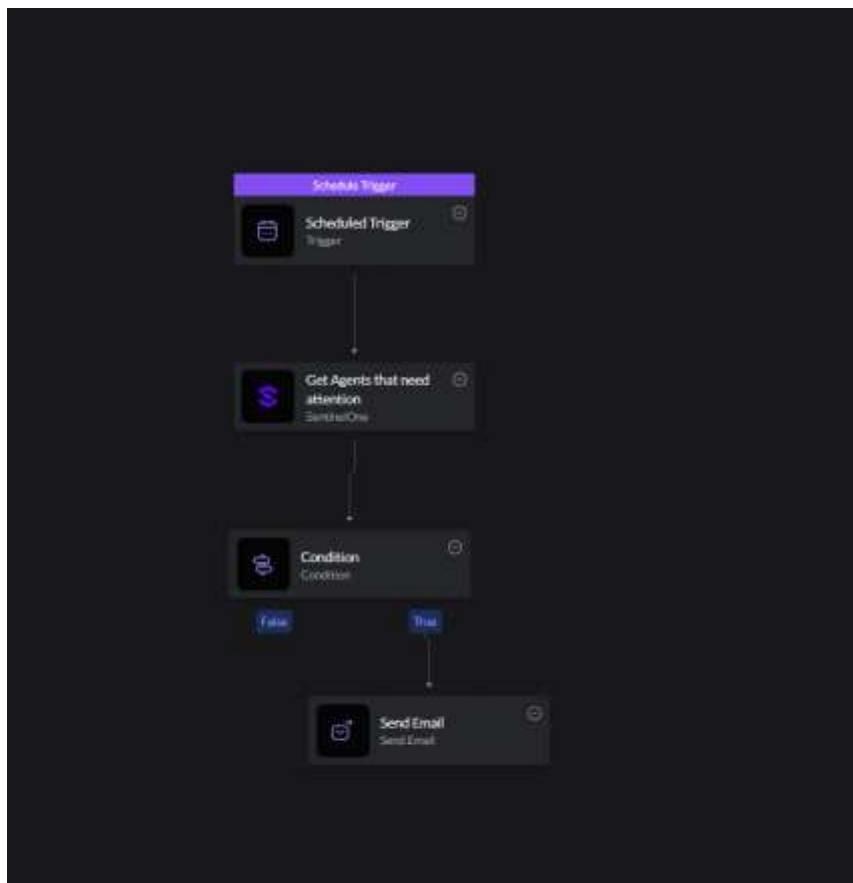


6.2.3. Workflow 3: Warn the client if agents are down. Json

De status van de SentinelOne-agent is het fundament van de beveiliging. Zodra een agent een foutstatus vertoont, verliest het team het zicht op de veiligheid van die pc. De flow Warn client if agents are down.json ondersteunt het beheer door via de API de status van de agenten op te vragen. Waar de logica voorheen keek naar hoe lang een computer offline was (bijvoorbeeld zeven dagen), is dit nu aangepast om ruis te voorkomen. Veel apparaten, zoals reserve-laptops of pc's van medewerkers met ouderschapsverlof, zijn immers vaak legitiem langere tijd offline zonder dat er een technisch probleem is. De flow richt zich nu uitsluitend op problematische cliënten die expliciet een foutstatus rapporteren. Onder een 'defecte' agent verstaan we een installatie waarbij de beveiligingsservices zijn gestopt of de

software corrupt is geraakt. In dat geval is de computer mogelijk wel actief, maar biedt de EDR-oplossing geen actuele bescherming meer.

Bij een gedetecteerd probleem gaat er automatisch een bericht naar de IT-beheerder van de klant. Deze melding bevat de hostnaam van de betreffende computer en de specifieke reden voor de foutstatus, zodat er direct gericht kan worden ingegrepen. Dit script bespaart het SOC-team veel handmatig werk; zij hoeven niet langer zelf dashboards te doorzoeken op zoek naar haperende agenten. Door deze proactieve werkwijze worden gaten in de dekking gedicht voordat aanvallers er misbruik van kunnen maken. Dit verhoogt de algemene weerbaarheid van de digitale omgeving van klanten die vertrouwen op de diensten van VanRoey (Dynamate) voor hun online beveiliging en bedrijfscontinuïteit

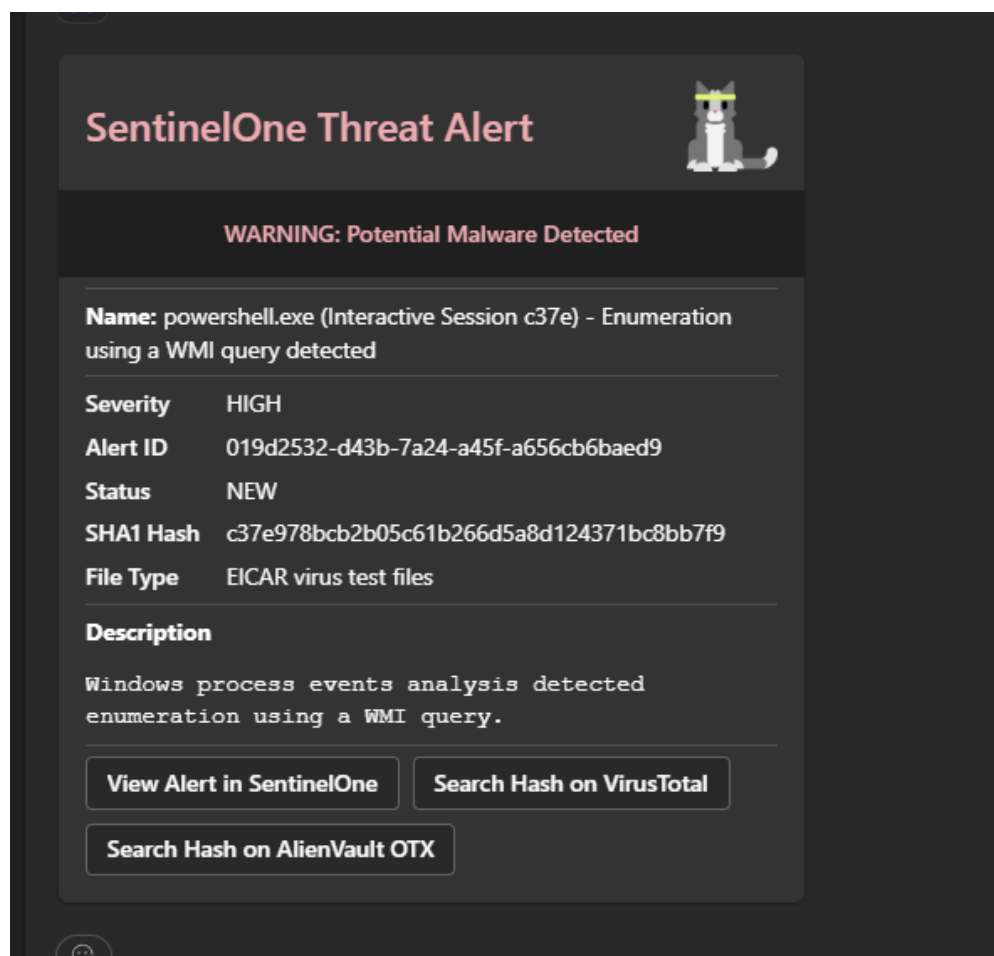


Name	Value
userActionsNeeded	user_action_needed,reboot_needed,upgrade_needed,incompatible_os,unprotected,rebootless_without_dynamic_detection,extended_exclusions_partially_accepted,reboot_required,pending_deprecation,ne_not_running,ne_cf_not_active,pending_performance_insights,reboot_category,missing_permissions_category,agent_suppressed_category,incompatible_os_category,unprotected_category,partial_functionality,performance_insights

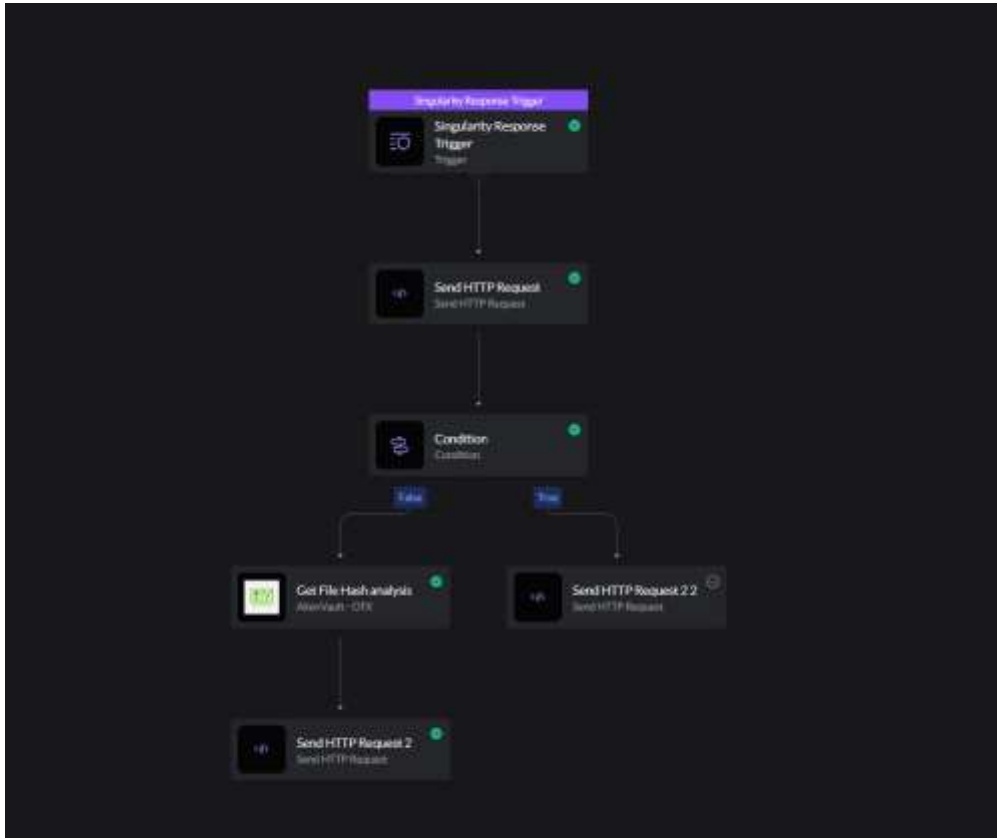
+ Add Parameter

6.2.4. Workflow 4: teams alerts. Json

Voor een goede 'Human-in-the-Loop' (HITL) moeten analisten alerts meteen zien. De `teams alerts. JSON-flow` zorgt voor de koppeling met Microsoft Teams. Dit script stuurt alerts door als de score hoog is. De kern is de 'Adaptive Card'. Dit is een JSON-stukje dat een bericht maakt in Teams met kleuren. Analisten zien zo in één oogopslag wat er mis is. In de kaart zitten knoppen voor de console van SentinelOne, het ticket in Autotask en Virustotal.



Dit zorgt voor een centrale stroom van meldingen zonder het dashboard steeds te moeten verversen. Het bouwen van deze stromen is heel belangrijk voor de HITL-opzet. Het stelt het team in staat om sneller te overleggen en gezamenlijk te besluiten welke stappen nodig zijn bij een ernstig incident. Zo wordt de samenwerking tussen de teamleden versterkt en de responstijd verder ingekort, wat de veiligheid van alle aangesloten systemen waarborgt. Deze slag is de sleutel tot een sneller SOC dat klaar is voor de toekomst en de groeiende hoeveelheid data die verwerkt moet worden door de analisten van VanRoey. Het systeem zorgt voor een filter dat alleen de meest relevante zaken doorstuurt naar de menselijke experts voor verdere afhandeling en onderzoek.



6.3. Uitdagingen binnen Si Hyperautomation

De scripts werkten in de testfase, maar het project liep tegen de technische grenzen van het low-codeplatform aan en de kosten van het platform waren hoog. Het principe van 'fail fast' was zinvol: we zagen de gebreken direct. De omgeving volstond niet voor de taken van een professioneel SOC. Hieronder leggen we uit waarom de software niet paste bij de behoeften van analisten van het team dat aan complexe automatisering werkt.

6.3.1. Complexiteit van JSON Manipulatie met JQ

Een groot probleem was JQ voor het aanpassen van data. Hyperautomation werkt bijna alleen met JSON. Als een API MISP-data geeft, is deze vaak onoverzichtelijk en diep genest in verschillende arrays. Om info uit een complexe response te halen, moesten we lastige JQ-codes schrijven in kleine vakjes van de interface. JQ is krachtig maar ook streng. Een klein foutje in de code, zoals een ontbrekend punt of een foute sluithaak, zorgde ervoor dat de hele flow stopte. De foutmeldingen waren vaak niet te begrijpen en boden geen hulp bij het debuggen. Ook het opvangen van fouten was een knelpunt. API-data zijn niet altijd hetzelfde. Soms mist er een veld of is een waarde nul. In Python kun je dit makkelijk oplossen met een controle. In JQ moest er veel if-then-else-logica in een regelcode om te

6.3.2. Technische Imitaties bij Privilege Escalation en Lateral Movement

In week 5 liepen we tegen een ander probleem aan met de Hyperautomation-omgeving tijdens het testen van de scripts met gesimuleerde aanvallen (zoals Privilege Escalation en Lateral Movement). De bedoeling was dat workflows specifieke technieken uit het MITRE ATT&CK-raamwerk zouden herkennen (zoals T1078 Valid Accounts of T1021 Remote Services), om daarna automatisch accounts te blokkeren en pc's te isoleren.

In de testomgeving gebeurde dit echter niet zoals verwacht. Bij acties zoals Lateral Movement greep de SentinelOne-agent lokaal op kernelniveau al in. De pc werd geïsoleerd voordat de Cloud Console de benodigde data kon doorsturen om de flow te triggeren. Omdat de trigger ontbrak, konden de workflows niet starten en was testen onmogelijk.

Een andere beperking van de flow-builder was het correleren van gebeurtenissen over een langere periode. Een aanval bestaat vaak uit meerdere fasen die zich over meerdere dagen afspelen. Hyperautomation verwerkt alerts als losse events en is in de basis stateless. Zonder het gebruik van een externe database was het in een visueel script niet mogelijk om een alert van PC 'A' van gisteren logisch te koppelen aan een alert van PC 'B' van vandaag.

De eerste vier gebouwde scripts functioneerden, maar voor verdere groei en beheerbaarheid op de lange termijn bleek de no-codeomgeving niet toereikend. Dit inzicht was de aanleiding om het project fundamenteel te wijzigen: we zijn gestopt met de grafische flow-builder en overgestapt op een zelfgebouwde Python-architectuur (zie Hoofdstuk 8). Door code te schrijven kregen we de controle over de logica en foutafhandeling die we nodig hadden voor de dagelijkse operaties binnen het SOC van VanRoey.

7. IMPLEMENTATIEFASE 3: VERKENNING VAN HET REWST PLATFORM

Dit hoofdstuk beschrijft waarom het project het Rewst-platform heeft onderzocht als alternatieve SOAR-oplossing (Security Orchestration, Automation, and Response). Ook komen de eerste workflows en de evaluatie van de low-code-automatisering aan bod. De tekst documenteert het proces dat tot de overstap naar Python-microservices heeft geleid.

7.1. Beperking van SentinelOne Hyperautomation

In de vijfde week van de stage bleek de initiële opzet met SentinelOne Hyperautomation niet haalbaar. Het eerste plan was om de automatiseringslogica in de EDR-omgeving (Endpoint Detection and Response) te bouwen, om dataverkeer met externe API's te beperken. Tijdens het testen van de scenario's liepen we echter tegen limieten aan.

Een probleem was het simuleren van aanvalsscenario's, zoals privilege-escalatie en lateral movement. De SentinelOne-agent op de test-endpoint (Windows- en Linux-machines in het Hyper-V-cluster) reageerde te snel. We probeerden deze aanvallen te simuleren om te testen of de SOAR-workflow de machine kon isoleren. Zodra er echter kwaadaardige acties werden gestart, stopt de behavioral AI van de EDR-module het proces onmiddellijk. De acties werden direct geblokkeerd en de flow kon daardoor niet getriggerd worden.

Hoewel dit de taak van een EDR is, maakte het de ontwikkeling van de playbooks onmogelijk. Je kan geen reactie automatiseren op een event dat door de preventielag wordt tegengehouden. Hierdoor was de validatie van `Human-in-the-loop` interacties onuitvoerbaar.

Daarnaast waren er integratieproblemen met het MISP-platform (Malware Information Sharing Platform). Aanhoudende HTTP 502- en 400010-foutmeldingen vanuit de SentinelOne-backend maakten de native MISP-connector onbruikbaar. We probeerden de communicatie op te zetten via GraphQL en REST-API's. De interface van Hyperautomation bood te weinig functionaliteit om de foutafhandeling en datatransformaties met JQ (JSON Query) in te stellen. Bij de verwerking van geneste Json-response crashte de visualisatiemodule van SentinelOne vaak. De gemaakte workflows waren daardoor onleesbaar en foutgevoelig voor de SOC-analisten die het beheer moesten overnemen.

Daarom keken we naar het Rewst-platform, een Robotic Process Automation (RPA)- en SOAR-platform voor managed service providers (MSP's). VanRoey beschikte over testlicenties voor dit systeem. In Rewst kunnen processen visueel worden geautomatiseerd met een workflow Builder Canvas, API-connectoren (Integrations) en de Jinja-templatetaal. In deze opzet is SentinelOne uitsluitend de sensor die de alert doorgeeft. Rewst verwerkt de waarschuwing, haalt informatie uit MISP, voert de mitigatie uit en maakt het ticket in Autotask aan.

7.2. Ontwerpfase en architectuur van de workflows

Voordat de implementatie in Rewst startte, maakten we een abstract ontwerp om overbodig werk te vermijden. In week zes is de logica van SentinelOne-processen omgezet naar stroomschema's. We gebruikten Unified Modeling Language (UML)-diagrammen om de stappen en de beslissingsbomen te visualiseren. Zo konden de Senior SOC-analisten de logica controleren zonder de interface van Rewst te hoeven leren.

We hebben vier hoofdworkflows ontworpen. De eerste workflow verrijkt waarschuwingen met data uit MISP en Alienvault OTX. De Tweede isoleert gebruikers en werkplekken bij gecompromitteerde inloggegevens (Compromised Users). De derde workflow is een controlesysteem voor endpoints waarvan de SentinelOne-agent offline is. De vierde workflow escaleert het 'High confidence Ransomware'-incident naar Microsoft Teams.

We selecteerde de "Compromised Users" workflow om de haalbaarheid van Rewst te testen. Dit scenario, vaak het gevolg van phishing, komt geregeld voor en vereist een directe reactie.

De architectuur van deze workflow had de volgende stappen. Eerst moest de SentinelOne-alert binnenkomen via een HTTPs Webhook trigger op de infrastructuur van Rewst. De documentatie van Rewst legt dit uit in de secties 'Use Webhook triggers' en 'Configure a Webhook trigger for CORS request'. De relevante gebruikersdata staan in geneste sturen zoals 'targetInfo.networkInterfaces'. Via 'Data aliases' en 'Input variables' is de inkomende datastroom in het workflow builder canvas gestructureerd.

Daarna werd gecontroleerd of de activiteit actie vereiste. De workflow filtert op de status 'True positieve' en een severity van 'High' of 'Critical'. In Rewst wordt dit ingesteld via taken in de 'SentinelOne integration setup' en de 'Microsoft Cloud Integration bundel', gebruikmakend van 'Core Actions' en 'Transformation actions' om de data aan te passen.

De laatste stap was de registratie van het incident in Datto Autotask. Dit ticket bevat de context, de SHA-256-hashes en een link naar de SentinelOne-console. Na het aanmaken stuurt het systeem een geautomatiseerd bericht naar Microsoft Teams. Hiervoor is gebruikgemaakt van pre-built templates zoals de '[REWST - PROCESS] PSA: Create Ticket' subworkflow uit de bibliotheek van Rewst.

7.3. Implementatie en de Datto Autotask API blokkade

De opzet in de Rewst-interface begon met de integraties. In het New workflow builder canvas voeg je acties toe die worden omgezet in taken met instelbare 'Task transitions'. De webhook-trigger voor de SentinelOne-waarschuwingen werkte direct. Rewst ontving de Json-payload, en met data-aliassen koppelden we velden zoals het endpoint-ID aan interne variabelen.

De problemen startten bij de integratie met het Datto Autotask-ticketsysteem. Een SOC-proces heeft alleen nut als het de gegevens goed wegschrijft in het registratiesysteem. Een correcte werking met Autotask was een randvoorwaarde.

We volgden de documentatie van de 'Datto PSA Integrations setup' en de 'Least privilege access guide for Datto's Autotask PSA Integrations'. Autotask vereist een API-gebruiker met de rol "API-user (System) (API-only)". Er worden een key en een secret gegenereerd, met als "Integration Vendor" de optie "Rewst - Automation".

De 'Datto PSA Integrations kit' en subworkflows 'Datto PSA: Create Ticket' bieden acties voor Autotask, zoals het aanmaken van tickets via de 'POST /V1.0/Tickets' endpoint. Deze acties werkten niet in de testomgeving.

Wanneer de workflow bij de Autotask-taak kwam, gaf het Rewst-dashboard foutmeldingen. Deze fouten, zoals "400 Bad Request" of "Unauthorized", waren generiek en lieten niet zien welk veld in de payload fout was. Autotask vereist een strikt datamodel met interne numerieke waarden voor velden zoals het bedrijfs-ID (CompanyID) en de wachtrij-ID (QueueID). Er zijn 'Rewst Master' subworkflows zoals 'PSA-Datto: Get prioriteit' en 'PSA-datto: Get Queues' om deze ID's op te vragen. Toch leidde een verkeerd verwerkte variabele tot de afwijzing van het ticket.

Samen met een interne API-specialist hebben we de instellingen in Datto Autotask nagekeken. De benodigde permissies (waaronder 'Create company Webhooks') stonden correct ingesteld. We stelden de JSON-payload handmatig op in Postman en verzonden

deze. Deze aanroep was succesvol, wat aantoonde dat de inloggegevens werkten. Dezelfde call faalde wanneer Rewst-backend deze verstuurde.

Dit maakte een beperking van low-codeplatforms zichtbaar. Omdat HTTP-request door Rewst wordt afgehandeld, konden we de onbewerkte binaire payload niet inspecteren. Er is geen mogelijkheid om een packet capture uit te voeren. Tools zoals de `Context Viewer` en logboeken over `Troubleshoot workflow execution and task results` toonden een abstractie van de fout. De black-box-werking van de connector maakte het debuggen lastig. In een Python-script kan een ontwikkelaar het verkeer inspecteren door `print(response.request.body)` uit te voeren. Deze diagnostiek ontbreekt in Rewst.

7.4. Kritische evaluatie van low-code voor SOC-operaties

Na de problemen met de Autotask-integratie hebben we de inzet van Rewst herzien. Platformen als Rewst richten zich op standaard MSP-processen. De handleiding besteedt veel aandacht aan de 'Microsoft Onboarding Solution Kit' en de 'Forms Portal'. Voor SOC-operaties, waar ongestructureerde malware-artefacten en threat intelligence worden verwerkt, had het platform nadelen.

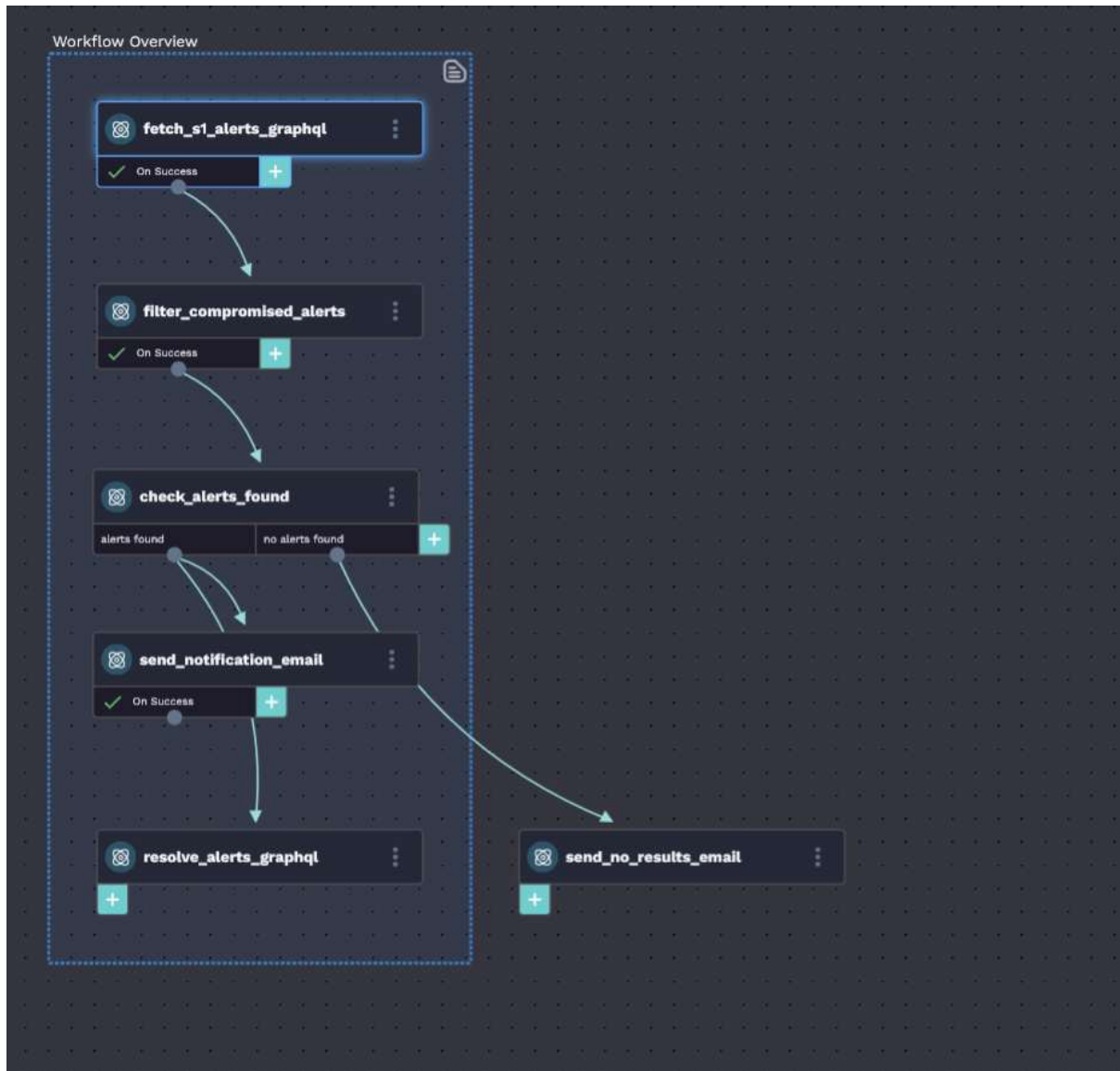
7.4.1. Jinja en abstractie van logica

Abstractie leidt in theorie tot snellere ontwikkeling doordat interacties verborgen worden gezet achter UI-elementen. Bij cybersecurity-workflows is controle over de onbewerkte data echter vereist. Dreigingsdata bestaat uit wisselende datumnotaties, arrays en soms nulwaarden.

Rewst gebruikt Jinja voor datatransformaties, beschreven onder 'Jinja essentials'. `Jinja-data types` en `Jinja filters`. Het toepassen van logica met iteraties ('loops in Jinja') of foutafhandeling in tekstvelden is foutgevoelig en lastig te debuggen. Als een veld leeg is wegens een ontbrekend IP-adres en de Jinja-expressie daar geen rekening mee houdt, stopt de workflow met een fout.

Rewst biedt ook 'transform action', waaronder 'Extract part of that transform action', 'Flatten list transform actions'. Het vertalen van code-concepten naar visuele blokken zorgt voor grote netwerken van acties op het canvas. Waar een Python-script een datum in een regel formatteert, vereist dit in Rewst meerdere verbonden stappen.

Als een pre-built actie faalt, is er geen mogelijkheid om lokaal een fix in de onderliggende code toe te passen, behalve door eigen requests via `Core actions` op te bouwen. Hierdoor ben je voor aanpassingen afhankelijk van de releasecyclus van de leveranciers.



```

{
  "name": "fetch_s1_alerts_graphql",
  "publicMetadata": "s1_graphql_response",
  "description": "Fetch compromised alerts via GraphQL - status NEW, scope to account, with full field set",
  "isLocked": false,
  "lockOwner": [],
  "with": {
    "type": "graphql",
    "url": "https://api.s1-alerts.com/graphql",
    "method": "POST",
    "headers": {
      "Content-Type": "application/json",
      "Authorization": "Bearer "
    },
    "body": {
      "query": "query { alerts { id category name type typevalue id } }",
      "variables": {
        "category": "category",
        "name": "name",
        "type": "type",
        "typevalue": "typevalue",
        "id": "id"
      }
    }
  },
  "timeout": 30,
  "transition": "success",
  "security": {
    "method": "POST",
    "params": {
      "url": "https://api.s1-alerts.com/graphql",
      "method": "POST",
      "headers": {
        "Content-Type": "application/json",
        "Authorization": "Bearer "
      },
      "body": {
        "query": "query { alerts { id category name type typevalue id } }",
        "variables": {
          "category": "category",
          "name": "name",
          "type": "type",
          "typevalue": "typevalue",
          "id": "id"
        }
      }
    }
  },
  "type": "graphql",
  "url": "https://api.s1-alerts.com/graphql",
  "method": "POST",
  "headers": {
    "Content-Type": "application/json",
    "Authorization": "Bearer "
  },
  "body": {
    "query": "query { alerts { id category name type typevalue id } }",
    "variables": {
      "category": "category",
      "name": "name",
      "type": "type",
      "typevalue": "typevalue",
      "id": "id"
    }
  },
  "timeout": 30,
  "transition": "success",
  "security": {
    "method": "POST",
    "params": {
      "url": "https://api.s1-alerts.com/graphql",
      "method": "POST",
      "headers": {
        "Content-Type": "application/json",
        "Authorization": "Bearer "
      },
      "body": {
        "query": "query { alerts { id category name type typevalue id } }",
        "variables": {
          "category": "category",
          "name": "name",
          "type": "type",
          "typevalue": "typevalue",
          "id": "id"
        }
      }
    }
  },
  "type": "graphql",
  "url": "https://api.s1-alerts.com/graphql",
  "method": "POST",
  "headers": {
    "Content-Type": "application/json",
    "Authorization": "Bearer "
  },
  "body": {
    "query": "query { alerts { id category name type typevalue id } }",
    "variables": {
      "category": "category",
      "name": "name",
      "type": "type",
      "typevalue": "typevalue",
      "id": "id"
    }
  },
  "timeout": 30,
  "transition": "success",
  "security": {
    "method": "POST",
    "params": {
      "url": "https://api.s1-alerts.com/graphql",
      "method": "POST",
      "headers": {
        "Content-Type": "application/json",
        "Authorization": "Bearer "
      },
      "body": {
        "query": "query { alerts { id category name type typevalue id } }",
        "variables": {
          "category": "category",
          "name": "name",
          "type": "type",
          "typevalue": "typevalue",
          "id": "id"
        }
      }
    }
  }
}

```

7.4.2. Snelheid en versiebeheer

Daarnaast waren er aandachtspunten voor het versiebeheer en de rekestijd. In SOC-toepassingen telt de reactiesnelheid bij incidenten. Een architectuur die elke stap in een gecentraliseerde cloudomgeving moet verwerken voegt latentie toe. Een lokaal gehost en asynchroon systeem presteert sneller.

De werking van `Workflow Builder` botst met `Infrastructure as code` standaarden. Rewst heeft `Crates` voor pre-built automation en `Document with RoboRewsty` voor documentatie. De Marketplace bevat modules zoals de Compromised user Response create`. Het beheren van deze componenten is gedocumenteerd in `Migrating between create versions` en `Use a create with a custom integration`. Als je een Crate aanpast, wijkt je af van de centrale repository, wat problemen geeft bij updates.

Binnen VanRoey gebruiken we Git voor het versiebeheer van scripts, wat code reviews mogelijk maakt. Wijzingen in een script kunnen worden nagekeken en teruggedraaid via standaardcommando's. In Rewst is de configuratie opgeslagen in de backend van de leverancier. Er zijn `organization and organization variables` en rolbankfuncties, maar de achterliggende code is een onleesbaar JSON-bestand. Dit maakt het moeilijk voor security audits en codebeoordelingen.

Er is functionaliteit om scripts in te laden via `Use Powershell scripts in Rewst` en de `Powershell interpreter`. Ook beschrijft de documentatie de inzet van `Agent Smith` (uitgerold via Intune of Immybot) om lokale taken uit te voeren. We hebben overwogen dit te gebruiken, maar dit introduceerde een extra agent op de eindpunten, wat overlapt met de reeds aanwezige SentinelOne-agent.

Bv. App access trigger SentinelOne:

The screenshot shows the Microsoft Sentinel interface for an alert titled "Azure Mailbox Permission Full Access As App Granted". The alert is categorized as "High" severity, detected by a "Platform Rule", and reported on "Apr 21, 2026 2:45:01 PM". The status is "New" and it is currently "Unassigned".

Alert Description and Recommendations: Detects when the fullaccess_as_app permission is assigned to an application, granting extensive access to all mailboxes within the organization. This permission allows the application to read, write, and manage mailboxes without user consent, posing a significant security risk. An adversary may exploit this to gain unauthorized access to sensitive emails and communications.

Rule Details:

Rule Name	Azure Mailbox Permission Full Access As App Granted	MITRE	Privilege Escalation (T1096)
Description	Detects when the full_access_as_app permission is assign...	Expiration Mode	Permanent
Scope	Account	Query	<pre>{dataSource.name = 'Azure Active Directory' and status='success' and activity_name = 'Add app role assignment to service principal' and unmapped.targetResources contains 'newValue':'\\full_access_as_app\\'} OR {dataSource.name = 'Azure Platform' and api.operation = 'Add app role assignment to service principal' and unmapped.properties.result='success' and unmapped.properties.targetResources contains 'newValue':'\\full_access_as_app\\'}</pre>
Severity	High		
Query Type	Single Event		
Query Language	2.0		
Data Source	Microsoft Entra ID Azure Platform		

Target Asset:

Target Name	service.rewst@vanrooy.be	Scope	Van Roey Automation - 020476 - MDR247/Site - Van R...
OS Type	Not reported		

7.4.3. Netwerkbeveiliging

Een ander punt was de netwerktoegang. De documentatie behandelt 'Incoming and outgoing domains and IPs' en de 'Webhook trigger rate limits' binnen het 'Rewst and security center'. Om de clouddienst te laten praten met on-premise server of afgeschermd interface (zoals interne MISP-servers en de Datto Autotask-tenant), moeten firewalls worden geopend voor externe IP-reeksen.

Het openzetten van verbindingen naar kernsystemen voor een externe applicatie vergroot het aanvalsoppervlak. Het instellen van multi-tenanttoegang via 'Permissions and roles' en 'App Builder: Authentication for apps' vereist bovendien administratief onderhoud. Een intern system bewaart alle API-verkeer en authenticatie sleutels binnen de eigen DMZ of cloud tenant, wat beter aansluit bij het interne beveiligingsbeleid

7.5. Overstap naar python

Na de problemen in de zesde en zevende week hebben we de testfase met Rewst stopgezet. Het projectdoel was het inkorten van de responstijd en het vermijden van afhankelijkheid van een enkele vendor. SentinelOne beperkte ons door de netwerkrestricties en de fouten in de MISP-integratie. Rewst had beperkingen bij de API-diagnostiek, de verwerking via Jinja en de integratie met GIT.

Daarom besloten we de applicatie-lag zelf te bouwen in plaats van het low-codeplatform te gebruiken. Het team kost voor custom microservices geschreven in Python.

Python beschikt over uitgebreide bibliotheken voor asynchrone HTTP-communicatie en het verwerken van complexe Json-data. Daar het script zelf te schrijven is, is het beheer onafhankelijk van ondoorzichtige modules in een cloudplatform. Als een API-aanroep in Python niet werkt, zijn de request-headers, de payload en de response te bekijken in de logs. Foutafhandeling van onvoorspelbare attributen of velden gebeurt met standaard programmeerstructuren in plaats van via workflow builder modules.

Na deze beslissing is de ontwikkeling van een eigen asynchrone infrastructuur gestart. Het nieuwe systeem gebruikt FastAPI voor de verwerking van inkomende webhooks, Rabbitmq en Redis om de taken in een wachtrij te plaatsen en Docker voor de containerisatie. De start van dit bouwproces wordt besproken in Implementatiefase 4.

8. IMPLEMENTATIEFASE 4: DE OVERSTAP NAAR PYTHON

In dit hoofdstuk wordt de technische realisatie van de asynchrone Python-architectuur beschreven. Dit volgt op de eerdere tests met SentinelOne Hyperautomation en Rewst. De secties behandelen de opzet van de microservices, de migratie van de vier workflows en aanpassingen aan het incidentmanagementproces.

8.1. Beperking van NO-code platformen en de keuze voor python

Tijdens de eerdere fasen van het project zijn SentinelOne Hyperautomation en Rewst geëvalueerd. Deze platformen maken gebruik van een visuele interface om processen te automatiseren. Bij de implementatie van de complexere use cases bleken er technische beperkingen te zijn. De beslissing om over te stappen op Python was gebaseerd op eisen rondom versiebeheer, foutopsporing en het verminderen van leveranciersafhankelijkheid.

8.1.1. *Versiebeheer en leesbaarheid van code*

Binnen een Security Operations Center worden automatiseringsscripts beheerd en gecontroleerd door meerdere analisten. In SentinelOne Hyperautomation worden JSON-berichten getransformeerd met behulp van JQ-filters. Voor het extraheren van specifieke waarden uit geneste arrays, zoals een MAC-adres uit een lijst van netwerkinterfaces, ontstonden er onoverzichtelijke JQ-expressies. In Rewst trad een vergelijkbaar probleem op bij het gebruik van Jinja-templates voor datatransformaties.

Daarnaast bieden visuele interfaces beperkte ondersteuning voor versiebeheer. Grote workflows met tientallen stappen zijn moeilijk te integreren in een standaard Git-workflow. Het vergelijken van wijzingen (diffing) tussen twee versies van een visuele workflow is in de gebruikersinterface veelal niet mogelijk. Dit compliceert het uitvoeren van peer reviews en het terugdraaien van foutieve aanpassingen via rollback-procedures.

Python is een programmeertaal die door de analisten in het SOC van VanRoey wordt gebruikt voor andere scripts. Door de playbooks in Python te schrijven, is de logica leesbaar in platte tekstbestanden. Dit maakt het gebruik van versiesystemen zoals Git mogelijk. Wijzigingen aan de playbooks verlopen nu via gecontroleerde pull requests, waarbij collega's de code kunnen inzien en goedkeuren voordat deze in de productieomgeving

wordt uitgerold. Ontwikkelaars kunnen documentatie direct bij de functies schrijven met behulp van Python-docstrings, wat de overdracht naar andere teamleden vereenvoudigt.

8.1.2. Foutopsporing en netwerkcontrole

Een tweede beperking van de low-codeplatformen betrof de foutopsporing bij inkomend en uitgaand netwerkverkeer. In Rewst verliep de communicatie met de Datto Autotask API via een ingebouwde connector, de zogenaamde Integration Bundle. Wanneer een HTTP-verzoek faalde met een HTTP-statuscode zoals “400 Bad Request”, toonde het platform een generieke foutmelding. Het was voor de gebruiker niet mogelijk om de exacte HTTP-headers, de binaire request body of de onbewerkte response van de Autotask-server te inspecteren.

Datto Autotask vereist een strak gedefinieerde JSON-structuur. Binnen de payload moeten specifieke numerieke waarden worden meegegeven voor velden zoals CompanyID, QueueID, IssueType en SubIssueType. Zonder inzage in de geconstrueerde payload was het troubleshooten van afgewezen API-verzoeken tijdrovend. Het platform fungeerde in dit opzicht als een afgesloten systeem (black box).

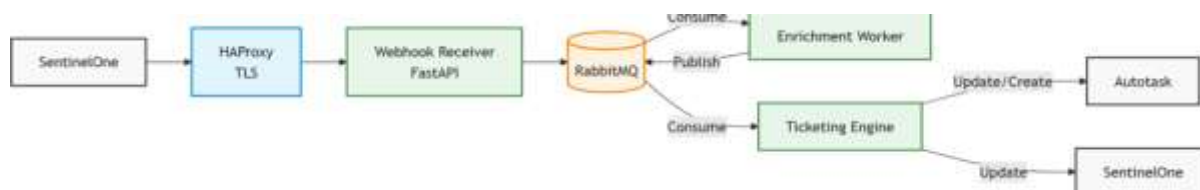
In een ontwikkelomgeving op basis van Python heeft de programmeur toegang tot het volledige netwerkverzoek. Met bibliotheken zoals ‘httpx’ kunnen payload en header voorafgaand aan verzending naar de console of een logbestand worden weggeschreven. Als een API-call mislukt, is in de stacktrace te herleiden welk dataveld de fout veroorzaakte. Python laat tevens toe om uitgebreide foutafhandeling (error handling) te schrijven, waaronder het inbouwen van ‘exponential backoff’-algoritmes wanneer een externe server tijdelijk een “429 Too Many Requests” response geeft.

Het gebruik van eigen code vermindert de afhankelijkheid van externe platformleveranciers. De gemaakte logica is niet meer gekoppeld aan de prioritaire interface van SentinelOne of Rewst. Bij een toekomstige migratie naar een andere Endpoint Detection and Response (EDR)-oplossing hoeft uitsluitend het inkomende datamodel in de Webhook-receiver herschreven te worden. De kernlogica voor dataverrijking, filtering en ticketcreatie blijft ongewijzigd.

8.2. Architectuur van python Microservices

De infrastructuur is geconfigureerd in de zevende en achtste week van het project. De applicaties draaien op gevirtualiseerde Linux-servers binnen een Hyper-V-cluster. De

codebase is opgesplitst in synchrone polling-scripts en asynchrone, event-driven microservices.



8.2.1. Polling Architectuur (standalone Scripts)

Voor taken die niet direct afhankelijk zijn van een real-time trigger, is een polling-architectuur opgezet. Hiervoor zijn standalone Python-scripts geschreven. Voorbeelden zijn het script dat controleert of SentinelOne-agents verbinding maken met de managementconsole, offline zijn of een reboot nodig hebben en het script dat actieve directory-audits uitvoert. Deze bestanden worden periodiek uitgevoerd door een cron-daemon of een Azure-Pipeline-taak. De scripts initiëren een verzoek aan de API, verwerken de ontvangen dataset en sluiten af na voltooiing.

8.2.2. Event-Driven Architectuur via Webhooks en FastAPI

Voor het verwerken van beveiligingswaarschuwingen (alerts) is een reactief, event-driven model geïmplementeerd. Dit deel van de architectuur bevindt zich in de Docker-setupmap van het project en is ondergebracht in afzonderlijke Docker-containers. De inkomende data stroomt binnen via een webhook-ontvanger, gebouwd met het FastAPI-framework.

FastAPI is geselecteerd vanwege de ondersteuning voor asynchrone bewerking via Python's `asyncio`-bibliotheek en de compatibiliteit met Pydantic voor datavalidatie. Zodra SentinelOne een malafide actie registreert, stuurt het EDR-systeem een HTTP POST-verzoek naar de FastAPI-endpoint. Met Pydantic-modellen wordt gecontroleerd of de ontvangen JSON-payload voldoet aan de verwachtingen, bijvoorbeeld of velden zoals `accountID` en `threatName` aanwezig zijn.

Tijdens eerdere testfasen werd de logica (zoals het bevragen van de MISP-server en het aanmaken van Autotask-tickets) sequentieel afgewerkt in de webhook. De verwerkingstijd liep hierdoor op tot vijf seconden per waarschuwing. Bij een stresstest met zeventig gelijktijdige meldingen overschreed de responstijd de drempelwaarde van SentinelOne. Dit leidde tot webhook-timeouts in de EDR-console. Omdat het systeem geen tijdig HTTP 200-antwoord ontving, markeerde SentinelOne de levering als mislukt en werden de berichten opnieuw aangeboden, wat de belasting op de server verder verhoogde.

[Link](#)

8.2.3. Message Brokering met Rabbitmq

Om de communicatie op te vangen en verwerkingspieken (spikes) glad te strijken, is Rabbitmq toegevoegd aan de infrastructuur. Het message-brokerpatroon scheidt het ontvangen van data van het verwerken van data.

De FastAPI-applicatie functioneert nu uitsluitend als ingresslaag. Zodra een POST-verzoek is gevalideerd door pydantic. Publiceert de server de payload onmiddellijk naar de Rabbitmq-exchange. Het script retourneert daarna binnen enkele milliseconden een http 200 ok-status aan SentinelOne, waardoor netwerk-time-outs worden voorkomen.

Rabbitmq plaatst de berichten in gedefinieerde wachtrijen. De asynchrone Python-workers luisteren via de `aio\_pika`-bibliotheek naar deze queues. Ze halen de berichten in hun eigen tempo op. Door gebruik te maken van berichtbevestigingen in Rabbitmq, wordt een bericht pas uit de wachtrij verwijderd nadat de workers de verwerking succesvol hebben afgerond. Als een worker crasht tijdens een netwerkverzoek, wordt het bericht automatisch teruggestuurd naar de wachtrij, zodat er geen data verloren gaat. De queues zijn geconfigureerd met disk-persistentie, zodat waarschuwingen behouden blijven bij een herstart van de Docker-container.



8.2.4. Deduplicatie en State Management met Redis

Malware kan op een geïnfecteerd systeem snel tientallen bestanden aanpassen.

SentinelOne genereert voor elk gedetecteerd proces of bestand een aparte waarschuwing.

Als de integratie voor iedere melding een nieuw ticket aanmaakt in Autotask, overspoelt dit het ticketsysteem en veroorzaakt er alert fatigue bij de analisten.

Voor het opslaan van de verwerkingsstatus en het voorkomen van ticket-deduplicatie is Redis ingezet. Redis opereert als een snelle in-memory key-value store. In de logica van de ticketing-worker wordt, voorafgaand aan een API-aanroep naar Autotask, verbinding gemaakt met de Redis-container. Er wordt gezocht naar een sleutel die is opgebouwd uit een samenstelling van factoren zoals het endpoint en de hash van malware.

Wanneer de sleutel niet wordt gevonden, verwerkt de code een nieuw ticket in Autotask. Het resulterende Autotask Ticket-ID wordt opgeslagen in Redis, voorzien van een Time-To-Live (TTL) van een vooraf bepaalde tijdsduur, bijvoorbeeld 24 h. Indien er binnen dit venster nieuwe waarschuwingen binnenkomen met dezelfde parameter, lezen de workers het opslagticket-ID uit Redis. In plaats van een nieuw object aan te maken, verstuurt het script een PATCH-verzoek naar Autotask om de aanvullende details als notities toe te voegen aan het initiële ticket.



8.3. Implementatie van Python playbooks

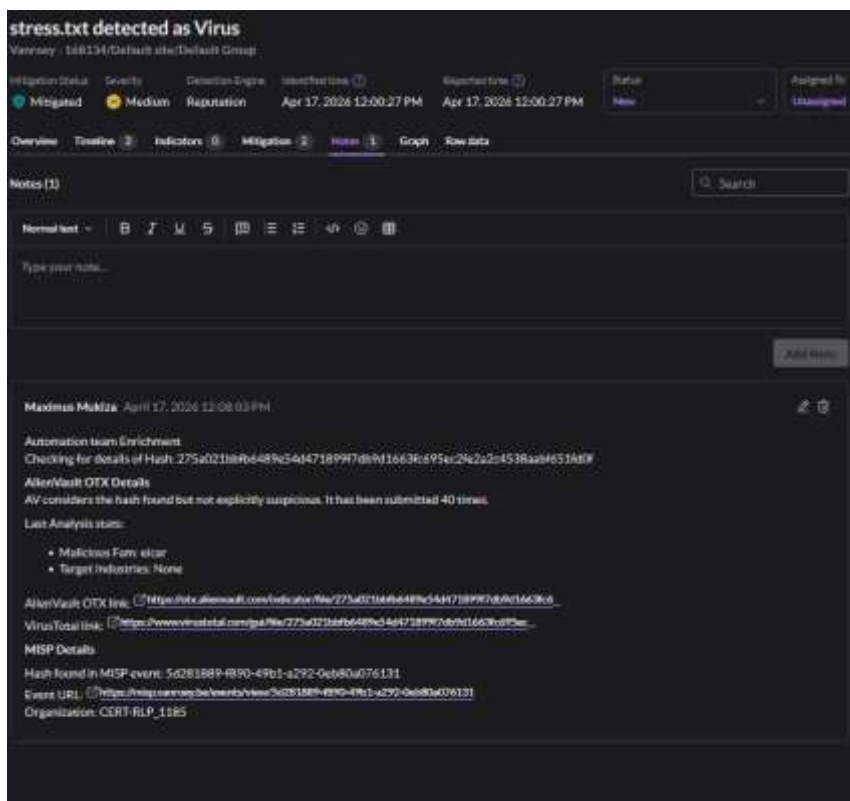
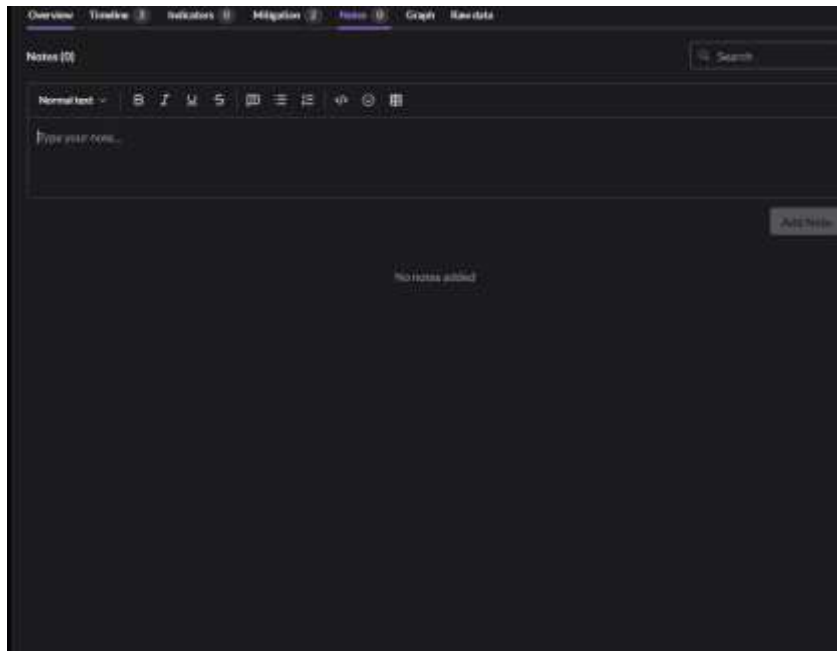
De automatiseringslogica is onderverdeeld in onafhankelijke Python-modules. In week 7 en 8 zijn deze ontworpen en getest. Voor uitgaande API-verzoeken maken de modules gebruik van `Httpx` om de event-loop van Python niet te blokkeren.

8.3.1. Waarschuwingen verrijken met Threat Intelligence (Enrichment workers)

De `enrichment\_worker.py`-module verwerkt de inkomende data uit Rabbitmq. De primaire functie is het toevoegen van dreigingsinformatie aan de kale waarschuwing. Het script onderzoekt de SentinelOne JSON-structuur op Indicators of Compromise. Met nadruk op SHA-1 en SHA-256 bestandshashes, IP-adressen en bestandsnamen.

Voor geselecteerde IoC's lanceren de workers parallelle API-verzoeken. Er wordt een POST-verzoek verzonden naar de endpoint van de MISP-server. Attriboot specificeert het type attriboot (md5 of sha256 of sha1) om te verifiëren of het bestand bekend is binnen de feeds van het centrum voor Cybersecurity of CIRCL. De teruggegeven data levert informatie over malwarecampagnes en behoort tot de Traffic Light Protocol (TLP)-classificatie.

Daarnaast raadpleegt het script de Alienvault Open Threat Exchange (OTX) via de OTX DirectConnect API. Dit endpoint levert 'pulses' op: rapporten en indicatoren gedeeld door de beveiligingsgemeenschap. Na ontvangst van beide gegevensstromen aggregeren de workers de gevonden tags en details, voegen deze toe aan een uitgebreid dictionaire-object en plaatsen dit object in de volgende wachtrij voor ticketing.



8.3.2. Script voor Gecompromitteerde Gebruikers (Credential audit)

Het bestand `s1\_credential\_audit.py` valt onder de polling-architectuur. Dit script verbindt met de SentinelOne GraphQL API. De gedefinieerde query is opgezet om specifiek `incident Types` op te vragen, waarbij de nadruk ligt op waarschuwingen met betrekking tot credential theft, phishing en verdachte inlogpogingen op werkplekken.

Het script parseert de respons om het gebruikersaccount (User Principal Name) te isoleren. Afhankelijk van de configuratie in het systeem voert de code een API-commando uit om de getroffen gebruiker of het apparaat te isoleren. Vervolgens bouwt het script een e-mailbericht op in HTML-formaat. Met de ingebouwde *smtplib-bibliotheek* maakt het script een verbinding met de SMTP-relay van VanRoey. Het bericht dat de accountdetails en de tijdstippen van de inbreuk bevat, wordt verzonden naar de verantwoordelijke contactpersonen.

Received Time	Incident Name	Severity	Integration ID	Target Asset	Platform	Vendor	Alert Status	Analysis ID	Assigned To	Classification	Has...
Mar 01 2024 5:00:00 PM	Compromised Password Detected	Medium	...	user01	Windows	SentinelOne	Resolved	...	...	...	...
Mar 01 2024 5:00:00 PM	Compromised Password Detected	Medium	...	svc.sentinelid	Windows	SentinelOne	Resolved	...	...	...	...
Mar 01 2024 5:00:00 PM	Compromised Password Detected	Medium	...	bad.pwd	Windows	SentinelOne	Resolved	...	...	...	...
Mar 01 2024 5:00:00 PM	Compromised Password Detected	Medium	...	Administrator	Windows	SentinelOne	Resolved	...	...	...	...

SentinelOne - Compromised Active Directory Credentials

Dear,

A scheduled credential audit performed by SentinelOne ISIDP has identified Active Directory accounts with passwords present in verified third-party data breaches. To eliminate the risk of credential stuffing and potential lateral movement, we recommend you to rotate the password for the following users:

User Principal Name

user01

svc.sentinelid

bad.pwd

Administrator

VanRoey SoC-team
support@vanroey.be

8.3.3. Controle van Agent status (Attention Monitor)

Voor het monitoren van de gezondheid van de SentinelOne-software is de 'S1_agent_attention_monitor.py'-module gebouwd. Het script maakt een GET-verzoek naar de agent-endpoint van SentinelOne. Het downloadt een actuele lijst van de geïnstalleerde agents en hun operationele status.

De code itereert over de resultaten en filtert de agents op statussen die beheer vereisen, zoals *Offline*, *Infected* of endpoints die wachten op een herstart na een software-update. Het script structureert deze data in een Python dictionary, gegroepeerd op het veld *accountId* of *siteId*. Dit resulteert in opgesplitste datasets per klant. Vervolgens formatteert het script per klant een incidentenrapport dat automatisch per e-mail naar de respectievelijke systeembeheerders wordt verzonden ter opvolging.

8.3.4. Tickets en notificaties Genereren (Ticketing Engine)

De 'ticketing_engine.py' worker luistert naar de RabbitMQ-wachtrij waar de verrijkte incidenten in terechtkomen. De taak van deze worker is het registreren van de meldingen in Datto Autotask en het waarschuwen van analisten in Microsoft Teams.

Voor Autotask construeert het script een JSON-payload die voldoet aan het ticketschema van de REST API. Interne mappings zetten de naam van de klant en de ernst van de waarschuwing om naar de juiste 'CompanyID' en 'Priority' integrer waarden. De payload wordt met een POST-request verstuurd naar de '/V1.0/Tickets' endpoint.

Voor de notificaties in Microsoft Teams stellen de workers een Adaptive Card samen. Dit is een Microsoft JSON-standaard voor het bouwen van interactieve interface-elementen in chatberichten. Het object wordt naar de team-Webhook-URL gepost. De gegenereerde kaart in het SOC-kanaal toont visuele indicatoren, de hostnaam van het endpoint, een samenvatting van Misp-data en hyperlinks.

[illegible]

8.4. Wijzigingen in de Operationele workflow

De overstap naar de Python-architectuur en de bijbehorende scripts heeft de workflow in het SOC beïnvloed. De applicaties automatiseren administratieve stappen die analisten voorheen handmatig in de interfaces uitvoerden. Bij het ontwerp is overlegd met SOC-analisten, waaronder Milan, Reno en Matthias, om de inhoud van de Datto Autotask-tickets af te stemmen op hun werkprocessen.

De gegenereerde Autotask-tickets bevatten nu specifieke identificatienummers en URL's. De Python-code extraheert het unieke incident-ID uit de inkomende SentinelOne-data en genereert een deeplink in de ticketomschrijving. Hierdoor navigeert de gebruiker direct van Autotask naar het specifieke detailvenster in het EDR-platform.

Het script zet bestandsindicatoren om in referentielinks. Een SHA-256-hashwaarde wordt automatisch samengevoegd met de basis-URL van Virustotal ('https://www.virustotal.com/gui/file/{hash}'). Dit neemt de stap weg waarbij de analist de indicator kopieert en handmatig een zoekopdracht uitvoert in beveiligingstools. De combinatie van de Redis-deduplicatie, de directe links en de geïmporteerde Threat Intelligence verlaagt het aantal benodigde interacties tijdens de initiële beoordeling van een incident. De microservices zijn aan het einde van de achtste week getest op de virtuele servers van VanRoey.

9. KWALITEITSBORGING, TESTEN EN VALIDATIE

In een operationeel Security Operations Center (SOC) heeft een programmeerfout directe gevolgen. Een onterechte netwerkislatie van een productieserver, veroorzaakt door een haperend script, verstoort de bedrijfsvoering van de klant enorm. Vanwege deze hoge risico's vereiste de Python-code een strikte testfase. De validatie verliep in verschillende stappen. Eerst kwam er een afgeschermd testomgeving. Vervolgens lag de focus op het genereren van realistische malware-waarschuwingen, de technische controle van de datastromen en ten slotte de formele acceptatie door de analisten. kwaliteitsborging

9.1. Opzet van de Virtuele Testinfrastructuur

Testen op live-klantnetwerken was vanzelfsprekend geen optie. Het project startte met de opbouw van een geïsoleerde sandbox.

9.1.1. *Hyper-V en Client Virtuele Machines*

Microsoft Hyper-V diende als basis voor deze lokale infrastructuur. Binnen de hypervisor kregen verschillende virtuele machines vorm. Denk hierbij aan standaard Windows-omgevingen, maar ook kale Linux-installaties. Elke virtuele endpoint ontving een eigen SentinelOne-agent. Deze agents mochten echter onder geen beding data sturen naar de actieve monitoring-dashboards van VanRoey. Een afgescheiden test-tenant in de EDR-console ving al het verkeer af.

9.1.2. *Backend Server en Containerisation*

De zwaardere backend-componenten kregen een plek op een aparte server-VM, beschikbaar gesteld door het interne IT-beheer. Docker-engine nam de taak op zich om de verschillende microservices te hosten. Via Docker Compose draaien Rabbitmq, Redis, FastAPI en de diverse Python-workers broederlijk naast elkaar. Door alles in containers te stoppen, bleef de omgeving op elk moment reproduceerbaar. Bovendien kon het netwerkverkeer vanuit de cloud-tenant de lokale webhook op deze manier veilig bereiken.

9.2. Simulatie van Dreigingen met EICAR en Ikarus

De code kwam pas in beweging bij inkomende beveiligingswaarschuwingen. Echte ransomware loslaten was onverantwoord, zelfs op een afgeschermd VLAN. Testbestanden boden een uitweg.

9.2.1. Gebruik van Testbestanden

Het befaamde EICAR-bestand genereerde de eerste stroom aan alerts. Zodra dit stukje tekst ergens de harde schijf raakte, greep SentinelOne in. De console pushte vervolgens een JSON-waarschuwing naar de FastAPI-server. Om de deduplicatie-algoritmes in Redis af te stellen, volstond één statisch bestand niet. De Ikarus-malware test set loste dat op door continu nieuwe bestandsnamen, paden en SHA-256 hashes te creëren.

9.2.2. Beperkingen bij het Testen van Gedragsdetectie

Naast losse bestanden stond er ook een test met PowerShell-scripts op de planning. Het doel was het nabootsen van Lateral Movement of Privilege Escalation. De EDR-software stak daar resoluut een stokje voor.

SentinelOne leunt zwaar op Behavioral AI en grijpt extreem vroeg in. Vrijwel meteen na de start van een script sloot de agent het achterliggende proces af. Het dashboard kreeg de status 'gemitigeerd' mee. En daar wrong de schoen: de SOAR-workflows luisteren puur naar onopgeloste incidenten. Om een aanval de tijd te geven zich te ontplooien, moesten de beveiligingsmodules deels uit. Dat vernietigde de realiteitswaarde van de test. Het zwaartepunt verschoof daarom definitief naar de statische Indicators of Compromise (IoCs).

9.3. Validatie van de Python Playbooks

De software vloog door meerdere controlefases. API-verbindingen kregen unit tests en de volledige datapijplijn liep tegen een integratietest aan.

9.3.1. Unit Testing en Datavalidatie

Postman nam de rauwe API-aanroepen voor zijn rekening. Bij Datto Autotask vormden de verplichte Basic Authentication en de API Integration Code-header de grootste struikelblokken. Postman hielp ook bij het in kaart brengen van de exacte structuur voor het POST-verzoek van een ticket. De SentinelOne API sprak GraphQL in plaats van REST, wat een heel ander type query vereiste om syntaxisfouten te vermijden.

Pas na goedkeuring belandden de calls in de Python-code als asynchrone httpx-requests. De Pydantic-bibliotheek fungeerde hierbij als strenge poortwachter. Zonder datamodel belanden foutieve datatypes (zoals tekst in plaats van getallen bij een QueueID) direct op de servers van Autotask. Dit resulteert onvermijdelijk in een Bad Request. Nu stopt de lokale validatie de verwerking ruim van tevoren.

9.3.2. *Integration Testing en Error Handling*

Integratietesten focusten op de route die de data aflegde. Van de FastAPI-ontvanger naar de message broker, om uiteindelijk bij de asynchrone workers uit te komen.

Foutafhandeling stond hierbij centraal. Wat gebeurt er als de MISP-server offline gaat? Een simpele test gaf antwoord. De container ging op slot, de enrichment_worker crashte tegen een ConnectTimeout aan en het Python-script sloeg aan via een try-except-blok. Er vertrok direct een Nack-commando (negative acknowledgment) naar Rabbitmq. Het bericht wachtte geduldig in de rij tot de Threat intelligence feed weer bereikbaar was. Het 'fail-open'-principe slaagde hiermee voor de test.

9.3.3. *Load Testing en Redis Deduplicatie*

Het deduplicatiealgoritme in Redis kreeg vijf exacte kopieën van een alert te verduren in een tijdsbestek van één seconde. De webhook ontving de stroom, verwerkte alert nummer één tot een volwaardig Autotask-ticket en parkeerde het bijbehorende ticket-ID in het geheugen met een limiet van dertig minuten. Bij de vier nakomers zag het systeem de specifieke endpoint-sleutel al staan. Een nieuwe POST-call bleef uit. De software stapelde de extra waarschuwingen slim op in het bestaande ticket door middel van HTTP PATCH-verzoeken.

9.4. *User Acceptance Testing (UAT)*

De echte lakmoesproef lag bij de eindgebruikers. SOC-analisten Matthias en Milan namen de User Acceptance Testing voor hun rekening. Hun voornaamste taak bestond uit het beoordelen of de tool daadwerkelijk een versnelling van hun workflow opleverde.

9.4.1. *Beoordeling van Ticket Kwaliteit*

Een simpele EICAR-test op een VM zette alles in gang. Nog geen dertig seconden later dook het eerste verrijkte ticket op in de wachtrij van de tweede lijn.

De ingebouwde Deep Link trok meteen de aandacht. Waar men vroeger lange apparaat-ID's overtipte in de SentinelOne-console, volstond nu één muisklik in de omschrijving. Ook het uiterlijk van de bestandshashes viel in de smaak. Het Python-script bouwde klikbare links

voor iedere gevonden SHA-256-waarde. Alienvault OTX en Virustotal zaten zodoende nog maar een seconde verwijderd van de analist. De conclusie luidde dat de initiële tijd voor triage stevig was ingekort.

9.4.2. Validatie van Notificaties (Microsoft Teams)

Als afsluiter richtte de UAT zich op de Teams-notificaties. De klassieke tekstberichten hadden plaatsgemaakt voor kleurrijke Adaptive Cards. Actieknoppen, hostnamen en dreigingsniveaus sprongen direct in het oog, wat het overzicht bevorderde.

Voor het specifieke script dat gecompromitteerde wachtwoorden opspoorde, verliep de waarschuwing via e-mail. De HTML-opmaak paste netjes in de huisstijl en de interne SMTP-server pikte de overdracht zonder problemen op. Met een formeel akkoord van het SOC kwam de testfase na acht weken tot een eind. De applicatie mocht officieel de operatie in.

10. RESULTATEN EN BUSINESS IMPACT

Dit hoofdstuk bespreekt de resultaten van het integratieproject. De belangrijkste doelen waren het verlagen van de Mean Time to Respond (MTTR), het minimaliseren van vendor lock-in en het automatiseren van incident triage. Na de overstap naar de Python-architecturen zijn deze doelen gehaald, met duidelijke gevolgen voor de werking van het SOC bij VanRoey.

10.1. Operationele Impact: Reductie van de Mean Time to Respond (MTTR)

Voor dit project moesten SOC-analisten inkomende alerts grotendeels handmatig triëren. Dit kostte veel tijd, wat bij incidenten zoals ransomware een probleem is.

De nieuwe Python-Microservices vangen SentinelOne-alerts direct op en verwerken ze asynchroon. Vroeger zochten analisten zelf in externe bronnen en moesten ze met slechte tickets leven. Momenteel gebeurt dit automatisch: het systeem maakt het ticket aan en haalt de dreigingsinformatie op. Dit proces duurt enkele seconden, wat de TTR verlaagt. De analist beoordeelt daarna alleen nog het mitigatieadvies via de human-in-the-loop flow. Hierdoor kan het SOC meer alerts verwerken zonder dat de werkdruk voor analisten evenredig stijgt.

10.2. Kwalitatieve Impact: verrijkte context en consistente Troubleshooting

Tijdens sprint 4 en 5 hebben we op basis van feedback van SOC-analisten (Matthias, Joey en Milan) extra informatie aan de tickets toegevoegd. Het doel hiervan was om de context van een alert duidelijker te maken.

De aangemaakte Autotask-tickets bevatten nu de volgende data:

- Een ID-referentie naar de SentinelOne-console
- Gegroepeerde waarschuwingen per klant of site, om dubbel werk te voorkomen
- Sha-links naar het platform zoals Virustotal en Alienvault
- Threat intelligence data from MISP

Omdat deze informatie direct in het ticket staat, hoeven analisten minder te schakelen tussen verschillende schermen. Dit maakt de troubleshooting eenduidiger en verkleint de kans dat een analist informatie zoals IP-reputatie over het hoofd ziet.

10.3. Architectural Impact: Elimination of vendor lock-in

Het was een vereiste om de afhankelijkheid van een enkele security-leverancier te verkleinen. In het begin van het project probeerden we integratie te bouwen via SentinelOne Hyperautomation en Rewst. Deze platformen veroorzaakten echter een sterke vendor lock-in. Bij problemen met de software, zoals de defecte MISP-connector in SentinelOne die niet opgelost raakte na contact met support, konden we zelf geen aanpassing doen.

Door over te stappen op een eigen Python-architectuur hebben we deze lock-in opgelost. We ontwikkelen de logica nu zelf met tools als pydantic en Rabbitmq en draaien deze in Docker-containers op de server van het bedrijf. Hierdoor controleert VanRoey de SOAR-functionaliteit zelf. Nieuwe threat intelligence-bronnen kunnen zonder tussenkomst van een externe partij worden toegevoegd. Mochten we in de toekomst een andere EDR-oplossing kiezen in plaats van SentinelOne, dan hoeft alleen het inkomende webhook-script aangepast te worden. De rest van de code blijft hetzelfde.

10.4. Financiële Impact: Python Script vs Licentiekosten van Rewst/Hyperautomation

De eigen Python-oplossing heeft een financieel voordeel. Platformen zoals Rewst en SentinelOne hanteren licentiemodellen die afhankelijk zijn van het aantal gebruikers of het aantal uitgevoerde acties. Omdat VanRoey als MDR-provider steeds meer endpoints beheert, zouden deze maandelijkse kosten snel oplopen, vooral bij workflows die veel data verwerken.

Met open-source technologieën op de bestaande servers vervallen deze licentiekosten voor de SOAR-integraties. De kosten bestaan nu enkel nog uit de ontwikkelingstijd en het beheer van de code. Omdat het SOC-team al bekend is met Python, is intern onderhoud goed mogelijk en zijn er geen dure externe consultants of speciale trainingen meer nodig. Hierdoor is de huidige infrastructuur makkelijker en goedkoper op te schalen.

11. CONCLUSIE

Dit hoofdstuk blijkt terug te grijpen op de oorspronkelijke probleemstelling en beantwoordt de onderzoeksvraag. Daarnaast volgt een reflectie op de iteratieve werkwijzen en de overstap naar een in-house ontwikkelde oplossing

11.1. Beantwoording van de onderzoeksvragen

De hoofdvraag van dit project was hoe Threat Intelligence en SOAR het beste konden integreren in de SentinelOne-omgeving van VanRoey, met als doel de triage-tijd te verkorten en leveranciersafhankelijkheid tegen te gaan.

De uiteindelijke geautomatiseerde pipeline heeft de tijd die nodig is voor incident triage aanzienlijk verlaagd. Het handmatig raadplegen van platforms zoals MISP en Virustotal is niet meer nodig, aangezien de Python Microservices deze dreigingsinformatie direct aan de Autotask-tickets toevoegen. De keuze voor een zelfgeschreven Python-architectuur, in plaats van een low-codeplatform, heeft de gewenste onafhankelijkheid opgeleverd. Het SOC-team kan zonder tussenkomst van externe partijen zelf API-wijzingen doorvoeren of nieuwe diensten aan het proces koppelen.

11.2. Algemene conclusie

De integratie heeft geresulteerd in een stabiele, asynchrone oplossing voor het SOC. De nieuwe infrastructuur vangt SentinelOne-alerts op, verrijkt deze met benodigde context en maakt Autotask-tickets aan. De toepassing van Rabbitmq als message broker zorgt ervoor dat het systeem goed blijft presteren, ook bij een hoog aantal gelijktijdige meldingen.

Binnen deze opzet functioneert MISP succesvol als het centrale platform voor Threat Intelligence. De koppeling met de datastromen van het CCB en CIRCL was al in week 2 gerealiseerd. De native integratie tussen MISP en SentinelOne bleek echter onwerkbaar door softwareproblemen bij de leverancier. Dit vormde een technische blokkade, maar dwong tegelijkertijd de overstap af naar een flexibeler en robuuster eigen systeem. De huidige in-house infrastructuur levert VanRoey meer beheermogelijkheden en lagere structurele kosten op dan commerciële SOAR-pakketten zouden doen.

11.3. Lessons Learned: De waarde van pivoteren in software development

Tijdens de uitvoering van de stage bleek flexibiliteit belangrijker dan het vasthouden aan een vaste planning. In de vierde week werd duidelijk dat de benodigde MISP-connector in SentinelOne Hyperautomation structurele gebreken vertoonde. Wachten op een oplossing vanuit de leverancier was geen optie binnen het tijdsbestek van dertien weken. Het plan werd daarom aangepast aan het gebruik van GraphQL en directe HTTP-verzoeken.

Een soortgelijke situatie deed zich voor bij Rewst. Dit platform leek een geschikte low-code oplossing, maar het troubleshooting van complexe API-fouten was nauwelijks mogelijk doordat het systeem onvoldoende logbestanden prijs gaf. De Agile-methodiek hielp om snel een beslissing te nemen: de low-code-route werd verlaten om de logica zelf in Python te programmeren.

Deze koerswijzigingen waren noodzakelijk voor het behalen van de projectdoelstellingen. Ze tonen aan dat technologie altijd ondersteunend moet blijven ten aanzien van het uiteindelijke doel. Door tijdig af te stappen van haperende externe platformen, kon er alsnog een werkend product worden afgeleverd dat goed aansluit op de technische kennis van de huidige SOC-analisten.

12. TOEKOMSTVISIE EN AANBEVELINGEN

De integratie heeft een werkende basis voor Security Orchestration, Automation, and Response (SOAR) opgeleverd binnen het SOC van VanRoey. De Python-microservices draaien in productie, maar het platform kan verder groeien. Dit hoofdstuk beschrijft de technische vervolgstappen en geeft advies aan het management om de infrastructuur op de lange termijn gezond te houden.

12.1. Verder uitbreiding van de python playbook bibliotheek

Tijdens deze stage zijn vier primaire workflows ontwikkeld, waaronder de verrijking van waarschuwingen en het monitoren van offline agents. De huidige python-architectuur is modulair, wat betekent dat ontwikkelaar makkelijke nieuwe scripts kunnen toevoegen zonder de bestaande code te breken

Het team kan deze bibliotheek uitbreiden met playbooks voor complexere aanvalspatronen. In de vijfde week bleek het bouwen van flows voor privilege-escalatie en lateral movement onmogelijk binnen SentinelOne Hyperautomation vanwege technische beperkingen van die tool. Nu we een eigen Python-omgeving hebben, vervallen deze obstakels. SOC-analisten kunnen scripts schrijven die direct ingrijpen in het Active Directory-domein of netwerkverkeer blokkeren via firewall-API's. Ik raad aan om periodiek te bekijken welke handmatige taken de meeste tijd kosten en deze als eerste om te zetten in nieuwe playbooks.

12.2. Toekomstige optimalisaties (OpenSearch/ElasticSearch integratie)

In de tweede week van het project onderzocht ik verschillende manieren om grote hoeveelheden data snel doorzoekbaar te maken. Daarbij kwamen Elasticsearch, OpenSearch en Typesense naar voren als mogelijke oplossingen.

Op dit moment vangen Redis en Rabbitmq de wachtrijen met inkomende SentinelOne-alerts goed op. Er ontbreekt echter nog een systeem om historische data over de lange termijn efficiënt te doorzoeken. Door OpenSearch of Elasticsearch toe te voegen aan de infrastructuur, krijgen analisten de mogelijkheid om oude logs en historische MISP-indicatoren in fracties van seconden te correleren met nieuwe dreigingen. Dit maakt Threat

Hunting gericht en zorgt ervoor dat het systeem snelle zoekresultaten blijft leveren naarmate het aantal klanten en logs toeneemt.

12.3. Advies voor het management van VanRoey

De overstap van commerciële low-code software naar eigen Python-code geeft VanRoey meer controle, maar stelt ook andere eisen aan de organisatie.

Allereerst is het aan te raden om de Python-kennis binnen het SOC-team (onder begeleiding van ervaren analisten zoals Matthias en Joey) te onderhouden. Omdat de broncode via Git wordt beheerd, is het proces van code reviews en versiebeheer goed te volgen. Het management doet er goed aan om structureel uren vrij te maken voor analisten om deze codebase bij te werken. Dit voorkomt dat de infrastructuur veroudert of afhankelijk wordt van één enkele werknemer.

Daarnaast raad ik af om zwaar te investeren in gesloten commerciële SOAR-platformen. De uitval van de MISP-connector in SentinelOne en de debugproblemen bij Rewst lieten zien dat leveranciersafhankelijkheid technische processen langdurig stil kan leggen. De huidige open-source oplossingen (Docker, FastAPI, Rabbitmq) werken stabiel en vereisen geen oplopende licentiekosten per gebruiker of endpoint.

Tot slot moet het goedkeuringsproces voor kritieke acties behouden blijven. Automatisering is effectief voor triage, maar het isoleren van een productieserver bij een klant brengt risico's met zich mee. Door het Human-in-the-loop (HITL)-mechanisme in stand te houden, voorkomt het SOC dat false positives leiden tot downtime bij klanten.

13. Literatuurlijst

Academische Publicaties en onderzoek papers

- Wagner, C., Dulaunoy, A., Wagener, G., & Iklody, A. (2016). MISP. ResearchGate, 49–56.
<https://doi.org/10.1145/2994539.2994542>
- Ruohonen, S., Kirichenko, A., Komashinskiy, D., & Pogosova, M. (2024). Instrumenting OpenCTI with a Capability for Attack Attribution Support. Forensic Sciences, 4(1), 12–23.
<https://doi.org/10.3390/forensicsci4010002>
- Sauerwein, C., Sillaber, C., Mussmann, A., & Breu, R. (2017). Threat Intelligence Sharing Platforms: An Exploratory Study of Software Vendors and Research Perspectives. Wirtschaftsinformatik und Angewandte Informatik.
- Roche, D., & Dowling, S. (2023). Elevating Cybersecurity Posture by Implementing SOAR. ResearchGate, 1–7.
<https://doi.org/10.1109/cyber-rci59474.2023.10671437>
- Mir, A. W., & Ramachandran, R. K. (2021). Implementation of Security Orchestration, Automation and Response (SOAR) in Smart Grid-Based SCADA systems. In Advances in Intelligent Systems and Computing (pp. 157–169).
https://doi.org/10.1007/978-981-16-1335-7_14

Frameworks en Standard

- Introduction to STIX. (n.d.). <https://oasis-open.github.io/cti-documentation/stix/intro.html>
- MITRE ATT&CK®. (n.d.). <https://attack.mitre.org/>
- Ocsf. (n.d.). GitHub - ocsf/ocsf-schema: OCSF Schema. GitHub.
<https://github.com/ocsf/ocsf-schema>
- Procida, D. (n.d.). Diátaxis. <https://diataxis.fr/>

Technische Documentatie en handleiding

- SentinelOne. (2024). Singularity Platform API Reference Guide. (Interne documentatie VanRoey/Dynamate).
- SentinelOne. (2024). Singularity Hyperautomation User Guide. (Intern documentatie VanRoey/Dynamate)

- SentinelOne. (2025). *The RoarinPenguin Guide to SentinelOne AI SIEM 2.3*. (Interne documentatie).
- Misp. (n.d.). MISP documentation and support. MISP Open-Source Threat Intelligence Platform & Open Standards for Threat Intelligence Sharing. <https://www.misp-project.org/documentation/>
- Datto. (2025b, September 29). Autotask PSA Software for IT Professionals | Professional Services Automation. <https://www.datto.com/products/autotask-psa/>
- Welcome to Rewst documentation | Rewst Documentation. (n.d.). Rewst Documentation. <https://docs.rewst.help/>
- RabbitMQ Documentation | RabbitMQ. (n.d.). <https://www.rabbitmq.com/docs>
- FastAPI. (n.d.). <https://fastapi.tiangolo.com/>

Website en Feed

- Helping to make Belgium the safest place to live & work online | CCB Belgium. (n.d.). <https://ccb.belgium.be/>
- CIRCL » Services. (n.d.). <https://circl.lu/services/>
- VirusTotal. (n.d.). VirusTotal. <https://docs.virustotal.com/>
- MalwareBazaar - Malware sample exchange. (n.d.). <https://bazaar.abuse.ch/>
- jq. (n.d.). <https://jqlang.org/>

14. BIJLAGEN

De onderstaande bijlagen bundelen de technische configuraties, broncode en operationeel Runbook van de ontwikkelde SOAR-Infrastructuur.

14.1. Bijlage A: Reference – API Endpoints, STIX Data Schemas' JSON workflows

14.1.1. *SentinelOne GraphQL Endpoints*

De vervanging van de defecte MISP-connector Vereiste direct toegang tot de SentinelOne GraphQL QPI ('/web/api/v2.1/unifiedalerts/graphql')

Voorbeeld GraphQL mutatie (Resolve en Notitie toevoegen)

```
mutation ResolveAndNoteAlert($id: String!, $note: String!) {
  alertTriggerActions(
    actions: [
      { id: "S1/alert/statusUpdate", payload: { status: { value: RESOLVED } } }
      { id: "S1/alert/addNote", payload: { note: { value: $note } } }
    ]
    filter: { or: [{ and: [{ fieldId: "id", stringEqual: { value: $id } }]} ] }
  ) {
    ... on ActionsTriggered {
      actions {
        actionId      success {
          id          }
        failure {
          id          }
        skip {
          id          }
      }
    }
  }
}
```

14.2. SentinelOne Hyperautomation (Legacy JSON Workflow)

Extractie uit de originele `Warn client if agents are down.json` workflow. Dit illustreert de “Scheduled Trigger”-logica en de API-aanroep via de standaard HTTP-connector.

14.2.1. HTTP Request configuration

```

''' JSON
{
  "name": "Get Agents that need attention",
  "action_type": "http_request",
  "method": "get",
  "url": "https://euce1-110-nfr.sentinelone.net/web/api/v2.1/agents",
  "parameters": [
    {
      "parameter_name": "userActionsNeeded",
      "parameter_value":
"user_action_needed,reboot_needed,upgrade_needed,incompatible_os,unprotected"
    }
  ],
  "headers": {
    "accept": "application/json",
    "Content-Type": "application/json"
  }
}
'''

```

14.2.2. Gekoppelde JQ Conditions

```

''' JSON
{
  "input_value": "{{Function.LENGTH(get-agents-that-need-attention.body.data)}}",
  "compared_value": "0",
  "comparison_operator": "not_equals"
}
'''

```

14.3. Bijlage B: Reference – Python Source Code van de Microservices

De Python-architectuur draait in Docker. Dit is de actuele code van de FastAPI Webhook-ingang en een fragment van de verrijgingslogica uit de Ticketing Engine.

14.3.1. Webhook (`soc\_scripts/docker/api/webhook\_receiver.py`)

```

''' python

```

```

from fastapi import FastAPI, Request, HTTPException
import logging
import datetime
from core.rabbitmq import publish_message
from core.config import settings

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

app = FastAPI(title="SentinelOne Webhook Receiver")

@app.on_event("startup")
async def startup_event():
    logger.info("Webhook Receiver started. Ready to accept payloads.")

@app.post("/")
async def receive_webhook(request: Request):
    """
    Receives JSON payload from SentinelOne Singularity Webhook.
    Instantly acknowledges with 200 OK and publishes to RabbitMQ.
    """
    try:
        import json
        payload = await request.json()
        alert_id = payload.get("id")

        logger.info(f"Received webhook for Alert ID: {alert_id}")

        if not alert_id:
            logger.warning("Received a webhook payload with no 'id'. Ignoring.")
            return {"status": "ok", "message": "No ID found."}

    except Exception as e:
        logger.error(f"Failed to parse JSON: {e}")
        raise HTTPException(status_code=400, detail="Invalid JSON payload")

    try:
        # Capture ingestion time
        ingestion_time = datetime.datetime.now(datetime.timezone.utc).isoformat()

        # Push external ID and ingestion timestamp to RabbitMQ
        message = {
            "externalId": alert_id,
            "_soar_ingestion_time": ingestion_time
        }
        await publish_message(settings.INGESTION_QUEUE, message)
        logger.info(f"Pushed alert {alert_id} with ingestion time {ingestion_time} to RabbitMQ.")
    except Exception as e:
        logger.error(f"Failed to publish to RabbitMQ: {e}")
        raise HTTPException(status_code=500, detail="Internal Server Error queueing message")

    return {"status": "ok"}

```

...

14.3.2. Autotask Ticketing Logica

(*'soc_scripts/docker/workers/ticketing_engine.py'*)

Structureren van de payload voor SOC-analisten in Autotask:

```

python
def format_alert_description(payload: dict, enrichment_details: str) -> str:
    """ Constructs a high-fidelity multiline description matching SOC standard formats. """ #
1. Core Identification & Timestamps
    alert_source = payload.get("detectionSource", {}).get("product", "SentinelOne")
    alert_name = payload.get("name", "N/A")
    alert_id = payload.get("id", "N/A")
    detected_at = payload.get("detectedAt", "N/A")
    ingestion_time = payload.get("_soar_ingestion_time", "N/A")
    # 2. Affected Entities
    asset = payload.get("asset") or {}
    detection_time_context = payload.get("detectionTime") or {}
    asset_context = detection_time_context.get("asset") or {}
    scope_context = detection_time_context.get("scope") or {}
    user = asset.get("lastLoggedInUser") or \
        detection_time_context.get("targetUser", {}).get("name") or "N/A"
    asset_name = asset.get("name", "N/A")
    ip_address = asset_context.get("ipV4", "N/A")
    # Asset Criticality Mapping
    asset_category = asset.get("category", "Workstation")
    criticality = "High" if asset_category == "Server" else "Medium"
    # Location Mapping
    account_name = scope_context.get("accountName", "N/A")
    account_id = scope_context.get("accountId", "N/A")
    site_name = scope_context.get("siteName", "N/A")
    group_name = scope_context.get("groupName", "N/A")
    location = f"{account_name}\t{site_name}\t{group_name}\t(ID: {account_id})"
    # 3. Indicators of Compromise (IOCs)
    process = payload.get("process") or {}
    file_context = process.get("file") or {}
    attacker_context = detection_time_context.get("attacker") or {}
    file_name = file_context.get("name", "N/A")
    file_path = file_context.get("path", "N/A")
    sha256 = file_context.get("sha256", "N/A")
    parent_process = process.get("parentName", "N/A")
    cmd_line = process.get("cmdLine") or "N/A"
    attacker_ip = attacker_context.get("ip", "None Detected")
    # S1 Console link
    s1_host = settings.S1_BASE_URL.replace("https://", "").replace("http://", "").rstrip("/")
    s1_link = f"https://{s1_host}/incidents/unified-alerts?alertId={alert_id}&viewType=all"

```

14.4. Bijlage C: How-To Guide – SOC Runbook: Applicatiebeheer & Docker

De applicatie wordt beheerd via Docker Compose op de server van VanRoey.

Stap 1: Applicatie opstarten

```

bash
docker-compose up -d --build

```

Configuratie in `docker-compose.yml` start vijf asynchrone processen:

1. `soc\_rabbitmq` (Message Broker)
2. `soc\_redis` (State Management & Deduplicatie)
3. `soc\_webhook\_receiver` (FastAPI op poort 8081)
4. `soc\_haproxy` (Reverse Proxy en SSL-terminatie voor poorten 80 en 443)
5. Python workers: `soc\_enrichment\_worker`, `soc\_ticketing\_engine`,
`soc\_credential\_audit\_worker`, `soc\_agent\_attention\_worker`.

Step 2: Foutopsporing in de logs

Bij het ontbreken van tickets controleer je de logs van de verantwoordelijke container:

```

` ``bash
# Inkomende requests inspecteren
docker logs -f soc_webhook_receiver

# Fouten in de Autotask ticket flow opsporen
docker logs -f soc_ticketing_engine
` ``

```

Stap 3: Configuratie en Credentials

API-sleutels (`S1\_API\_TOKEN`) staan gedefinieerd in het `.env`-bestand één map hoger (`../.env`). Herstart de containers na wijzigingen.

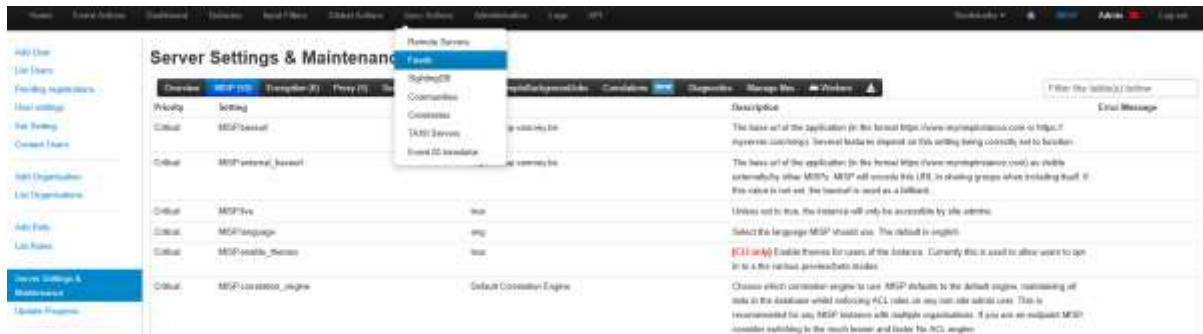
Bijlage D: How-To Guide – MISP Server Beheer en CCB/CIRCL Feed Synchronisatie

De integratie met externe Threat Intelligence vereist authenticatie bij CERT-dataroom-instances.

Instellen CIRCL OSINT Feed:

. Log in op MISP met adminrechten.

2. Navigeer naar ****Sync Actions**** -> ****Feeds****.



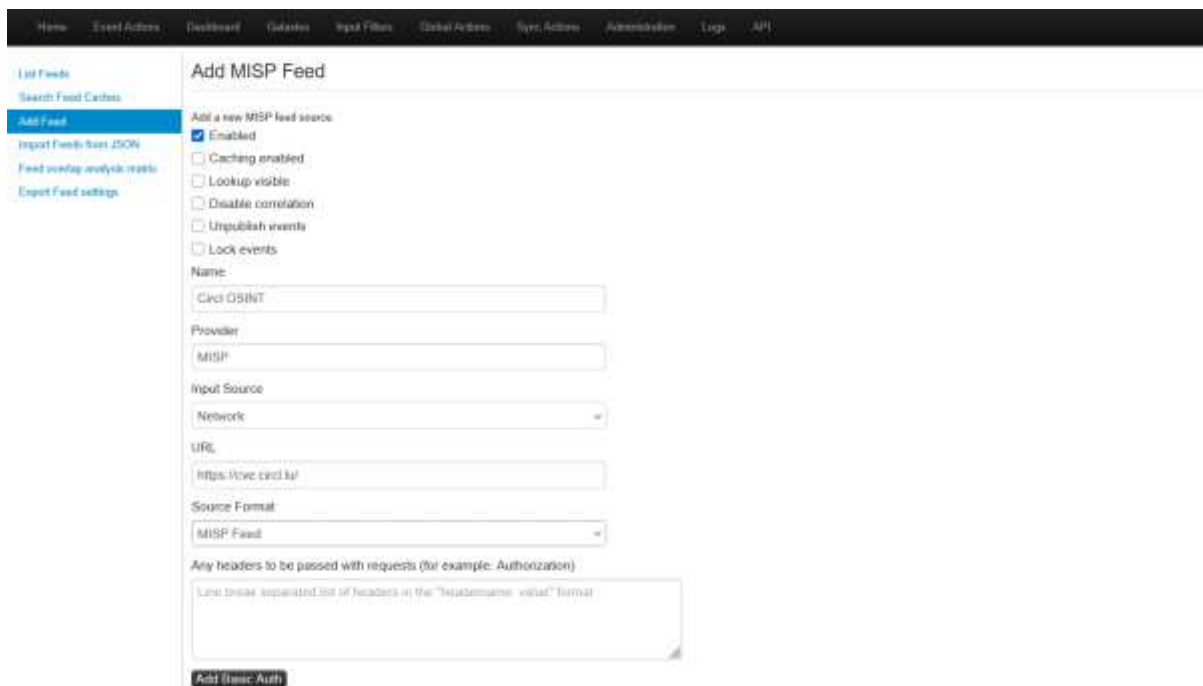
3. **Selecteer** → Add Feed.

4. **Naam:** CIRCL OSINT.

5. **Provider:** MISP.

6. **URL:** <https://cve.circl.lu/>

7. Markeer als ****Enabled**** en klik op **Fetch and Store data**



Verbinding met het Centrum voor Cybersecurity België (CCB)

1. Genereer een PGP/GPG sleutelpaar op de Linux-server of op windows met die tool:

<https://www.gpg4win.org/>

```
``bash
```

```
gpg --gen-key
```

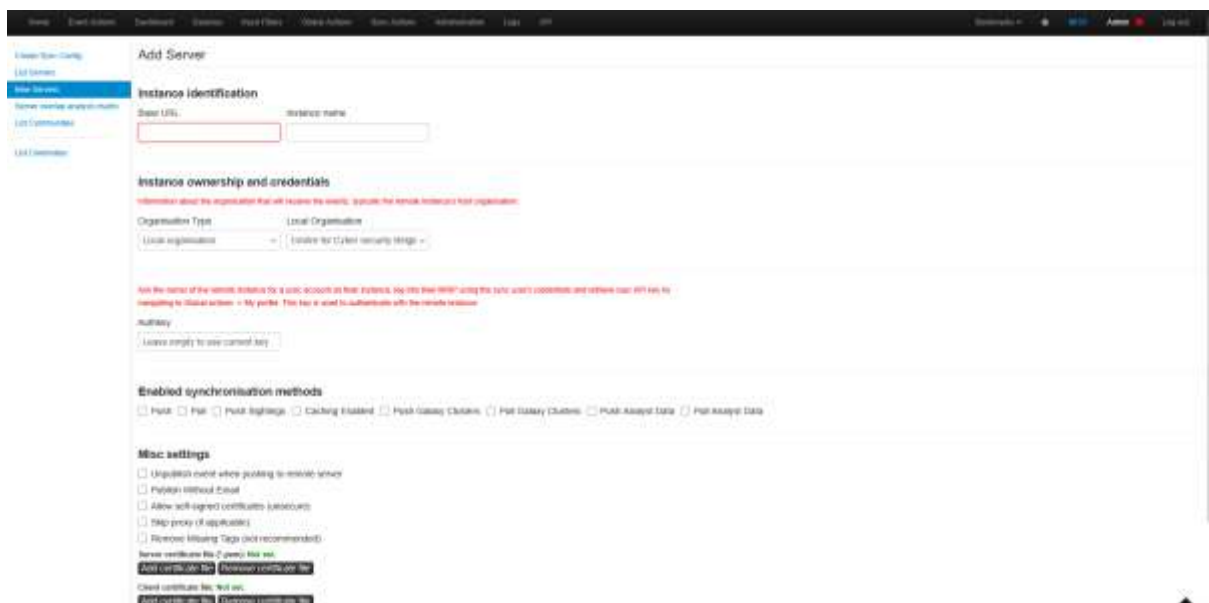
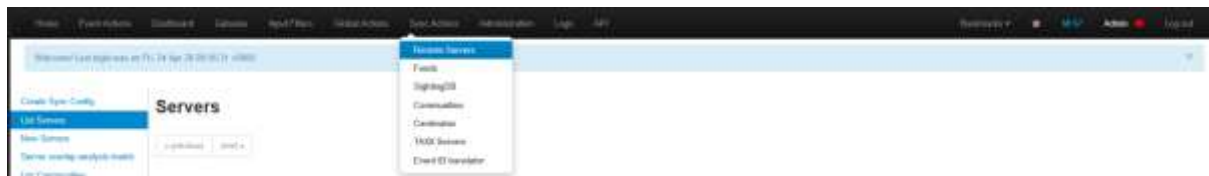
2. Exporteer de publieke sleutel.

```
```bash
```

```
gpg --export -a "Jouw Naam" > public.key
```

3. Mail de publieke sleutel naar het CCB om toegang tot de MISP-sync-server aan te vragen.

4. Voeg de CCB-server toe via **\*\*Sync Actions\*\*** -> **\*\*Servers\*\*** met de verstrekte URL en API-key.



## 14.5. Bijlage E: Explanation – Transcript en Analyse van het gefaalde SentinelOne Support Ticket

Tijdens sprint 2 leverde de native MISP-applicatie in de SentinelOne Singularity Marketplace fouten op in de testomgeving.

### Probleembeschrijving:

Bij het triggeren van de MISP-connector toonde de SentinelOne console de volgende foutmelding in de audit logs:

```
`"Integration Action Failed: HTTP 502 Bad Gateway - MISP Connector Response Error"``
```

### De technische blokkade:

Netwerkverkeersanalyse (vastgelegd in `failedMispIntegration.md`) toonde aan dat SentinelOne een STIX 2.1 JSON-object pushte. Bij afwezigheid van specifieke velden in SentinelOne, zoals registry keys, crashte de backend-STIX-parser en leverde een ongeldig patroon op. MISP weigerde dit met "HTTP 400 Bad Request".

### Reactie R&D:

SentinelOne R&D bevestigde het ontbreken van error-handling voor lege datavelden in de integratie. Een patch stond op de roadmap zonder vastgelegde releasedatum. Deze blokkade forceerde de overstap naar een eigen API-implementatie in Python.

### Bijlage F: Origineel Projectplan (v3) en YAML Architectuur

Het originele projectplan (`Project-plan\_v3.txt`) legde de fasering vast. De gebouwde Docker-architectuur komt overeen met de onderstaande services in de YAML-configuratie.

**Fragment van `docker-compose.yml`**

```
```yaml
services:
  rabbitmq:
    image: rabbitmq:3-management
    container_name: soc_rabbitmq
    ports:
      - "127.0.0.1:5672:5672"
      - "127.0.0.1:15672:15672"
    networks:
      - soc_network

  haproxy:
    image: haproxy:2.8-alpine
    container_name: soc_haproxy
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro
    depends_on:
      - webhook-receiver
      - rabbitmq
    networks:
      - soc_network
      - misp_default

  webhook-receiver:
    build: .
    container_name: soc_webhook_receiver
    command: uvicorn api.webhook_receiver:app --host 0.0.0.0 --port 8081
```
```

Dit YAML-configuratiebestand vormt de blueprint van de draaiende infrastructuur en integreert de berichtbroker, webhook en reverse proxy.